



Tunisian Republic

Ministry of Higher Education and Scientific Research

Private School of Engineering and Technology

TEK-UP

END-OF-SEMESTER PROJECT REPORT

Submitted in partial fulfillment of the requirements for

End-of-Semester Project

Specialty: Data Science and AI

Prepared by

Ahmed Klabi

Mariem Smadhi

Hichem Sboui

News Insight: An MLOps Platform for Tunisian News Analysis

Academic supervisor:

Sawssen Jalel

Academic Year 2025-2026

Contents

List of Acronyms	viii
General Introduction	1
1 Introduction	2
1.1 Introduction	3
1.2 Project context	3
1.3 Problem statement	3
1.4 Objectives	4
1.5 Work methodology	4
1.5.1 Iterative tracks	4
1.5.2 Version control workflow	5
1.6 Technology stack overview	5
1.7 Conclusion	7
2 Global Architecture	8
2.1 Introduction	9
2.2 High-level architecture	9
2.3 Actors and use cases	11
2.4 Domain model	11
2.5 Key scenarios	12
2.5.1 Scrape, transform, and load pipeline	12
2.5.2 AI-assisted semantic search	13
2.5.3 Ask-the-news chat (retrieval-augmented generation)	14
2.6 Conclusion	15
3 Data Ingestion and Web Scraping	16
3.1 Introduction	17
3.2 Scraping technology choice	17
3.3 Scraper architecture	17
3.4 Kapitalis scraper	19
3.5 RSS ingestion	20
3.6 Deduplication and raw storage	21

3.7	Error handling	22
3.8	Conclusion	23
4	Storage Layer and Database Design	24
4.1	Introduction	25
4.2	Database technology choice	25
4.3	Schema design	26
4.4	Vector storage with pgvector	28
4.5	Migrations	30
4.6	Conclusion	30
5	Machine Learning and NLP Pipeline	32
5.1	Introduction	33
5.2	Pipeline overview	33
5.3	Text embeddings	34
5.4	Sentiment analysis	35
5.5	Topic modeling with BERTopic	36
5.6	Enrichment: themes, labels and regions	37
5.7	Pipeline metrics	39
5.8	Conclusion	39
6	GenAI Features	41
6.1	Introduction	42
6.2	LLM provider abstraction	42
6.3	Article summaries	43
6.4	LLM topic labels and themes	44
6.5	AI semantic search	45
6.6	Ask-the-news chat assistant	46
6.7	Cost and reliability considerations	47
6.8	Conclusion	48
7	Orchestration and Experiment Tracking	49
7.1	Introduction	50
7.2	Orchestration technology choice	50
7.3	Flow design	51
7.4	Scheduling	53

7.5	Experiment tracking with MLflow	54
7.6	Monitoring and reliability	56
7.7	Conclusion	57
8	Backend and Data Exposure	59
8.1	Introduction	60
8.2	Backend technology choice	60
8.3	API structure	61
8.4	Endpoints	62
8.5	Data access patterns	65
8.6	Security and reliability considerations	66
8.7	Conclusion	67
9	Deployment and DevOps	68
9.1	Introduction	69
9.2	Containerization strategy	69
9.3	Service composition	70
9.4	Environment configuration	72
9.5	Data persistence	73
9.6	Developer workflow	74
9.7	Conclusion	75
10	Frontend and Data Visualization	77
10.1	Introduction	78
10.2	Frontend technology choice	78
10.3	Application structure	79
10.4	Article exploration and search UX	81
10.5	Visualizations and source diversity	82
10.6	Floating chat assistant	85
10.7	Conclusion	86
11	Final Conclusion and Future Improvements	87
11.1	Final conclusion	88
11.2	Project contributions	88
11.3	Future improvements	89
11.4	Closing remarks	91

List of Figures

1.1	Technology stack of the News Insight platform, grouped by pipeline stage: collection, storage, machine learning, orchestration/tracking, and application.	6
2.1	High-level architecture of News Insight: sources, storage, enrichment, orchestration/tracking, API, and frontend.	10
2.2	UML use case diagram: Visitor and Operator interactions with News Insight.	11
2.3	Domain model: Article and its conceptual Topic/Theme, Region, and Summary relations.	12
2.4	Sequence diagram: the scrape / transform / load pipeline, orchestrated by Prefect. . .	13
2.5	Sequence diagram: AI-assisted semantic search, with a keyword + LLM-rank fallback.	14
2.6	Sequence diagram: the ask-the-news RAG chat assistant.	15
3.1	Data ingestion flow: the HTML listing path (KapitalisScraper) and the RSS path (RSSScraper) both produce Article objects, deduplicated and stored through the same load step.	19
4.1	Storage layer schema: articles (central table, with its pgvector embedding column and indexes), entities (one row per extracted named entity, foreign key to articles), and pipeline_metrics (one row per pipeline run).	26
5.1	The ML/NLP pipeline: per-article transform (embeddings, sentiment, theme, region) feeding articles , and the separate, periodic BERTopic clustering stage that reads embeddings back out and overwrites topic_id/topic_label	34
6.1	The “ask the news” chat assistant (ChatWidget.jsx): a RAG answer over five retrieved articles, with clickable source citations.	47
7.1	Orchestration map: how a run starts (cron deployment or manual HTTP trigger), the news-pipeline flow and its single retried task, the run_pipeline steps it wraps, and the two observability endpoints (the Prefect UI and the pipeline_metrics table) covered later in this chapter.	51
7.2	Prefect UI’s Runs view, showing a news-pipeline flow run in progress, with its single task run and elapsed duration.	53
7.3	MLflow tracking UI showing a logged clustering run: parameters, metrics, and the fitted model artifact.	56

8.1	FastAPI interactive API documentation (Swagger UI) listing the platform’s endpoints.	65
10.1	The AI-assisted semantic search view (Search.jsx): each result carries a sentiment badge, source, title, and relevance score.	82
10.2	The home reader view (Reader.jsx , route /): the hero card for the latest article, followed by the source-interleaved “Dernières actualités” grid and its per-source filter chips.	84

List of Acronyms

- **ANN** = Approximate Nearest Neighbor
- **API** = Application Programming Interface
- **ASGI** = Asynchronous Server Gateway Interface
- **CORS** = Cross-Origin Resource Sharing
- **CPU** = Central Processing Unit
- **CSS** = Cascading Style Sheets
- **ETL** = Extract, Transform, Load
- **GIN** = Generalized Inverted Index
- **GPU** = Graphics Processing Unit
- **GenAI** = Generative Artificial Intelligence
- **HDBSCAN** = Hierarchical Density-Based Spatial Clustering of Applications with Noise
- **HTML** = HyperText Markup Language
- **HTTP** = HyperText Transfer Protocol
- **JSON** = JavaScript Object Notation
- **LLM** = Large Language Model
- **MLOps** = Machine Learning Operations
- **NLP** = Natural Language Processing
- **ORM** = Object-Relational Mapping
- **RAG** = Retrieval-Augmented Generation
- **RSS** = Really Simple Syndication
- **SPA** = Single-Page Application
- **SQL** = Structured Query Language

- **SSRF** = Server-Side Request Forgery
- **SVG** = Scalable Vector Graphics
- **UI** = User Interface
- **UML** = Unified Modeling Language
- **URL** = Uniform Resource Locator
- **WAF** = Web Application Firewall
- **WSGI** = Web Server Gateway Interface
- **XML** = Extensible Markup Language

General Introduction

Digital news consumption in Tunisia has grown steadily, but the outlets that produce it (*La Presse*, *Business News*, *Kapitalis*, *Mosaïque FM*, and others) remain as fragmented online as they are in print: each publishes independently, mixes French with Arabic-language content, and offers no shared index. A reader who wants a cross-source view of the day's news, or an analyst who wants to track how a topic or a region is covered over time, has no single place to look and no automated way to search, summarize, or ask questions of that content.

News Insight addresses this gap with an MLOps platform built around three verbs: *scrape*, *analyse*, *explore*. It continuously collects articles from multiple Tunisian sources, enriches them with multilingual embeddings, sentiment analysis, and unsupervised topic modelling, and adds a generative-AI layer (summaries, human-readable topic labels, and a retrieval-augmented "ask the news" chat assistant) on top. The whole pipeline is orchestrated, tracked, and containerized so it can be run and reproduced end to end, not just as a one-off analysis. The platform is developed as an End-of-Semester Project in Data Science and AI, academic year 2025-2026, by a three-person student team.

This report is organized into eleven chapters. Chapter 1 sets out the project's context, problem statement, objectives, work methodology, and technology stack. Chapter 2 presents the global architecture, including the use case, class, and sequence views of the system. Chapters 3 and 4 cover the data layer: web scraping and ingestion, then the PostgreSQL and pgvector storage design. Chapter 5 details the machine learning and NLP pipeline (embeddings, sentiment, BERTopic topic modelling), and Chapter 6 the GenAI features built on top of it (summaries, topic labels, semantic search, and the chat assistant). Chapter 7 describes how the pipeline is orchestrated with Prefect and tracked with MLflow. Chapter 8 documents the FastAPI backend that exposes the data, Chapter 9 the Docker-based deployment, and Chapter 10 the React frontend through which the platform is explored. Chapter 11 closes the report with a summary of the project's contributions, its current limitations, and the improvements planned for future iterations.

INTRODUCTION

1.1 Introduction

This chapter sets the scene for the rest of the report. It presents the context in which News Insight was designed, states the problem the platform addresses, lists the project’s objectives, describes the work methodology followed by the team, and closes with an overview of the technology stack used to build the platform. The chapters that follow (Global Architecture through Final Conclusion) each expand on one layer of the system that is only introduced here.

1.2 Project context

Tunisia has a dense but fragmented online news landscape. Outlets such as *La Presse*, *Business News*, *Kapitalis*, and *Mosaïque FM* each publish independently, in French and, for some outlets, Arabic, with no shared index and no cross-outlet view of what is being reported. A reader who wants a broad picture of the day’s news in Tunisia (what topics dominate, how sentiment shifts on a given subject, which region a story concerns) has to visit each site separately and build that picture manually.

News Insight is an academic MLOps project that turns this fragmented landscape into a queryable dataset. It continuously collects articles from multiple Tunisian sources, enriches them with natural language processing (embeddings, sentiment, topic modelling), exposes the result through an API and a web dashboard, and adds a generative-AI layer (summaries, topic labels, and a conversational "ask the news" assistant) on top. The project is carried out as an End-of-Semester Project in Data Science and AI, academic year 2025-2026, by a team of three students (Ahmed Klabi, Mariem Smadhi, and Hichem Sboui) under the supervision of Sawssen Jalel.

1.3 Problem statement

Beyond the lack of a single entry point, three concrete gaps motivate this project:

- **No aggregation.** Tunisian news outlets are not indexed together anywhere; comparing coverage of the same event across sources requires manual cross-referencing.
- **No semantic access.** Existing outlet websites only offer keyword search on their own archive, if that; there is no way to search across sources by meaning (e.g. find every article about a topic regardless of the exact wording used) or to ask a natural-language question and get a sourced answer.
- **No structured signal.** Raw articles carry no machine-readable topic, sentiment, or geographic tag, so tracking how a subject is covered over time, or which regions are mentioned, cannot be

done without reading every article by hand.

News Insight is built specifically to close these three gaps for the Tunisian news ecosystem, using an MLOps pipeline that keeps the data current rather than a one-off dataset.

1.4 Objectives

The project pursues four concrete, complementary objectives:

- **O1: Automated collection.** Continuously ingest articles from several Tunisian news sources without manual intervention, using RSS feeds as the primary channel and a dedicated HTML scraper as a fallback for deeper crawling of a single source.
- **O2: NLP enrichment.** Turn raw article text into structured signal: multilingual sentence embeddings for semantic retrieval, sentiment classification, and unsupervised topic discovery (BERTopic) so articles can be grouped, compared, and filtered automatically.
- **O3: GenAI access.** Let a user interact with the corpus in natural language: on-demand article summaries, human-readable topic labels, and a retrieval-augmented "ask the news" chat assistant that answers questions with cited sources.
- **O4: Reproducible MLOps.** Wrap the pipeline in tooling that makes it observable and repeatable rather than a collection of ad hoc scripts: workflow orchestration, experiment tracking for the clustering model, and a containerized deployment that any team member can bring up with a single command.

A fifth, supporting objective ties the previous four together: expose all of the above through a web dashboard so the results are usable by someone who does not read code. This is the subject of Chapter 10.

1.5 Work methodology

The team followed an agile, iteration-driven process suited to a three-person academic team rather than a formal heavyweight methodology.

1.5.1 Iterative tracks

Rather than designing the whole system up front, the work was organized into successive tracks, each building on the previous one and each shipped as soon as it was usable end to end: a *foundation* track (project scaffolding, database schema, Docker Compose stack), a *data sources* track (the scraper

base class, the RSS ingestion pipeline, source-specific quirks such as image and category extraction), an *ML/MLOps* track (embeddings, sentiment, BERTopic clustering, Prefect orchestration, MLflow experiment tracking), and finally an *application* track covering the GenAI features and the React frontend. This ordering is visible directly in the commit history: early commits scaffold scrapers and the database, middle commits add clustering and MLflow logging (e.g. *feat(mlops): log clustering runs + register model to MLflow*), and later commits build the chat assistant, semantic search, and UI on top of the by-then-stable pipeline (e.g. *feat(api): RAG chat over articles via pgvector*, *feat(ui): ask-the-news chat, sentiment timeline, article TL;DR*).

1.5.2 Version control workflow

The team used GitHub flow: every change of substance is developed on a short-lived feature branch, opened as a pull request against `main`, and merged once it is buildable. The repository (github.com/ahmedkltn/news-mlops) shows this pattern consistently across its history. Branches are named by intent, for example:

- `feat/news-hub`, `feat/tunisia-region-map`, `feat/ai-search-themes-diversity`: new features;
- `fix/ai-search-precision`, `fix/summary-thin-content`, `fix/map-projection-and-search`: bug fixes.

These branches were merged as pull requests #1 through #11 at the time of writing. Commit messages follow the Conventional Commits convention (`feat`, `fix`, `chore`, `docs`, `test`, `perf`, `ci` prefixes), which keeps the history readable and made it straightforward to reconstruct the project timeline for this report. A continuous integration gate (*ci: run pytest + frontend build on push*) runs the backend test suite and builds the frontend on every push, catching regressions before a branch is merged. Several pull requests in the log are targeted bug fixes discovered through use of the running system rather than planned features, for instance `fix/region-panel-scroll` (a scrollable list overflowing the page) and `fix/spa-cache-headers` (a stale single-page-app bundle being served from cache), which reflects the iterative, test-and-fix rhythm of the later part of the project.

1.6 Technology stack overview

News Insight is built as a small, coherent set of open-source tools, one per concern in the pipeline: collection, storage, machine learning, orchestration and tracking, and the application layer. Figure 1.1 shows the logo of each component; Table 1.1 summarizes the role each one plays. Later chapters revisit each of these choices in the context of the layer they belong to.

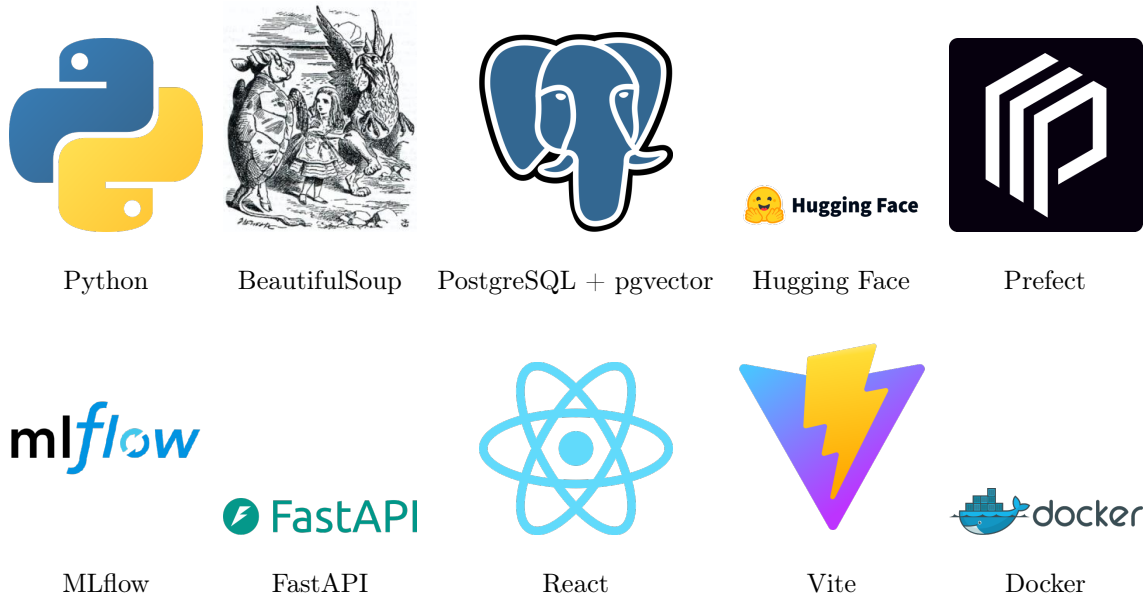


Figure 1.1: Technology stack of the News Insight platform, grouped by pipeline stage: collection, storage, machine learning, orchestration/tracking, and application.

Layer	Technology	Role
Collection	Python, BeautifulSoup	RSS parsing and HTML scraping of Tunisian news sources
Storage	PostgreSQL, pgvector	Relational storage of articles plus vector similarity search on their embeddings
Machine learning	Hugging Face (sentence-transformers, BERTopic, cardiffnlp models)	Multilingual embeddings, sentiment classification, unsupervised topic modelling
Orchestration	Prefect	Scheduling and running the scrape / transform / cluster pipeline
Experiment tracking	MLflow	Logging clustering runs and pipeline metrics for monitoring
API	FastAPI	Exposing articles, search, and GenAI endpoints to the frontend
Frontend	React, Vite	Web dashboard for browsing, searching, and chatting with the news corpus
Infrastructure	Docker, Docker Compose	Containerized, single-command deployment of the whole stack

Table 1.1: Role of each technology in the News Insight stack.

The GenAI layer (article summaries, topic labels, the "ask the news" chat assistant) is

deliberately not tied to a single vendor: it goes through an OpenAI-compatible abstraction so the underlying large language model provider can be swapped by configuration, with a free-tier provider used by default. Chapter 6 details this design choice.

1.7 Conclusion

This chapter framed the "why" of News Insight: a fragmented Tunisian news landscape with no unified analytical view, addressed through four objectives (automated collection, NLP enrichment, GenAI access, and reproducible MLOps) pursued through an iterative, GitHub-flow-based process. The remainder of the report follows the pipeline these objectives describe, starting with the global architecture that ties the stack introduced in this chapter into a single system.

GLOBAL ARCHITECTURE

Chapter outline

1	Introduction	3
2	Project context	3
3	Problem statement	3
4	Objectives	4
5	Work methodology	4
6	Technology stack overview	5
7	Conclusion	7

2.1 Introduction

Chapter 1 introduced News Insight’s objectives and the technology stack chosen to meet them. This chapter assembles those pieces into a single system view. It first walks through the high-level architecture, from the news sources on the left to the React dashboard on the right, then details three complementary UML views: the use case diagram (who does what), the class diagram (what the system stores), and three sequence diagrams that trace the concrete request flows behind the platform’s core features. Later chapters each zoom into one box of the diagrams introduced here: Chapters 3 and 4 detail the sources/storage boundary, Chapter 5 the ML enrichment step, Chapter 6 the GenAI layer, Chapter 7 the orchestration and tracking box, Chapter 8 the API, Chapter 9 the deployment, and Chapter 10 the frontend.

2.2 High-level architecture

Figure 2.1 shows the system end to end. Data flows left to right through five stages, with two supporting services (Prefect and MLflow) wrapped around the middle of the pipeline.

Sources. Two collection mechanisms feed the pipeline. `RSSScraper` polls the RSS feeds of four Tunisian outlets, *La Presse*, *Business News*, *Kapitalis*, and *Mosaïque FM* (`etl/extract.py`, list `RSS_FEEDS`), and is the active default for every source. `KapitalisScraper`, a `BeautifulSoup`-based HTML listing scraper built on the common `BaseScraper` class (`scrapers/base.py`), is kept available as a deeper-crawl fallback for `Kapitalis` but is not wired into the default extraction run. Both paths converge on the same `Article` record shape (`url`, `source`, `title`, `content`, `language`, `categories`, `published_at`, `image_url`) before anything is written to the database.

Storage. A single PostgreSQL instance, with the `pgvector` and `unaccent` extensions enabled, holds the whole corpus in one `articles` table (`db/schema.sql`). `pgvector` lets the same table carry a 384-dimension sentence embedding per row and be queried by vector distance; `unaccent` lets the same table be queried by accent-insensitive French full text. Centralizing storage this way (rather than splitting raw and enriched data into separate tables) keeps the enrichment step a set of in-place column updates instead of a multi-table join.

Enrichment. From storage, four enrichment operations read and write back to the `articles` table: multilingual sentence embeddings (`intfloat/multilingual-e5-small`), sentiment classification (`cardiffnlp/twitter-xlm-roberta-base-sentiment`), unsupervised topic clustering (`BERTopic`), and LLM-based tagging for theme and region. In the running system these are not four independent parallel jobs: embeddings, sentiment, and theme classification happen together, per article, inside `transform_articles` (`etl/transform.py`), and region is resolved by a `gazetteer`

lookup of Tunisian governorate names in the same step (an LLM-based resolver, `tag_region_llm`, exists in `etl/regions.py` but is disabled by default for cost and latency). BERTopic clustering is the exception: it needs a batch of already-embedded articles to fit a topic model, so it runs as a separate, on-demand stage (triggered by an operator rather than on every new article) that overwrites `topic_id/topic_label` with a data-driven clustering once at least ten articles are available. Chapter 5 details each of these models and their trade-offs.

Orchestration and tracking. Prefect wraps the scrape-transform-load sequence in a single flow, `news-pipeline` (`flows/news_flow.py`), whose one task retries twice with a 60-second delay on failure. The flow can be triggered on demand from the operator dashboard (POST `/pipeline/trigger`) or run on a daily schedule (06:00, Africa/Tunis, `flows/schedule.py`). Every BERTopic run is separately logged to MLflow under the `news-clustering` experiment (`etl/cluster.py`): parameters (embedding model, minimum cluster size), metrics (number of topics found, outlier percentage), and the fitted model itself as an artifact. Chapter 7 covers both services in detail.

API and frontend. A FastAPI application (`api/main.py`) exposes the enriched corpus through five routers (pipeline, articles, search, genai, and img) described in Chapter 8. Two of them, `search` and `genai`, call out to an external LLM provider (Groq by default, reached through an OpenAI-compatible client, as introduced in Chapter 1) for AI-ranked search fallback, article summaries, and the chat assistant. A React single-page application, built with Vite and served through nginx in production, is the only client of this API; nginx proxies every `/api/` request straight to the FastAPI container (Chapter 9, Chapter 10).

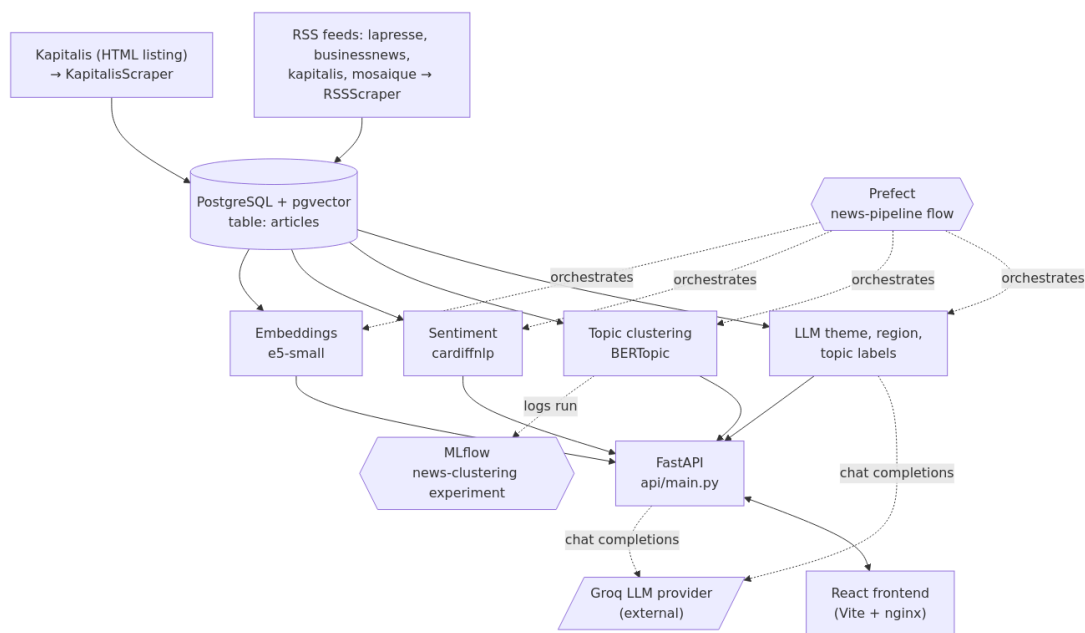


Figure 2.1: High-level architecture of News Insight: sources, storage, enrichment, orchestration/tracking, API, and frontend.

2.3 Actors and use cases

Two actors interact with the platform (Figure 2.2). The **Visitor** is anyone browsing the public dashboard: they can browse the article list and read a single article (`Articles.jsx`, `Reader.jsx`, `GET /articles`), run a semantic search (`Search.jsx`, `GET /search`), ask a question of the “ask the news” assistant (`ChatWidget.jsx`, `POST /genai/chat`), and view the topic, sentiment, and region dashboard (`Dashboard.jsx`, `Carte.jsx`). The **Operator** is a team member operating the pipeline: in addition to every visitor use case, they can trigger a new collection run or a clustering run (`Pipeline.jsx`, `POST /pipeline/trigger` and `POST /pipeline/cluster`) and monitor past pipeline runs from the same page’s status panel. The distinction is one of intent rather than of authentication (both roles use the same public frontend), but it separates the platform’s two concerns cleanly: consuming the enriched news corpus, and operating the pipeline that produces it.

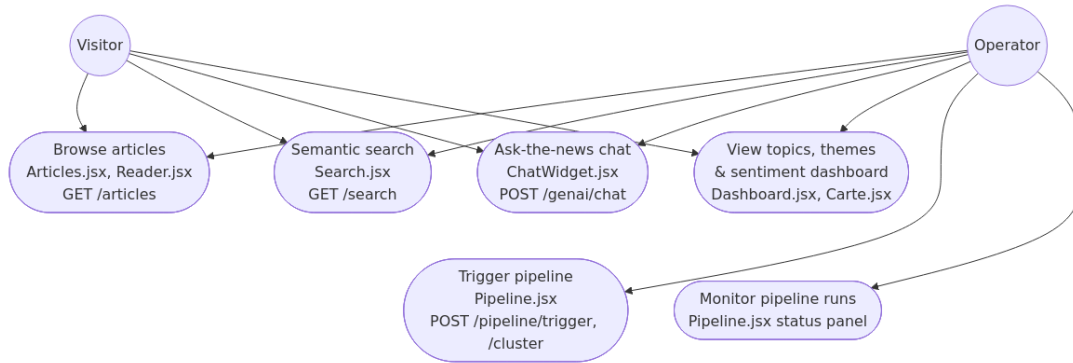


Figure 2.2: UML use case diagram: Visitor and Operator interactions with News Insight.

2.4 Domain model

Figure 2.3 gives the conceptual class view behind the single `articles` table. **Article** is the central entity: it carries source metadata (url, source, title, content, language, timestamps, image, categories) and the enrichment fields written by the pipeline described above: the 384-dimension embedding, the sentiment label, and `story_id` (reserved for a future cross-source story-clustering feature, not yet populated by the pipeline). Three further concepts relate to an article but are not separate database tables: **Topic/Theme** (a topic id and a human-readable label, set either by the LLM theme classifier or overwritten by BERTopic clustering), **Region** (the Tunisian governorate an article is tagged with, exposed on the map dashboard), and **Summary** (the on-demand, cached LLM summary produced by `GET /genai/summary/{id}`). Each of the three is modelled as a zero-or-one relationship from Article because each maps directly onto a nullable column set on the `articles` table (`topic_id/topic_label`, `region`, `summary`) rather than a foreign-key relation, a deliberate flattening that keeps every read a single-table query, at the cost of not normalizing region or topic metadata. The full schema also defines

two operational tables outside this conceptual model, an `entities` table for named-entity extraction and a `pipeline_metrics` table for run monitoring, covered where they are used in Chapters 4 and 7.

Topic/Theme, Region and Summary are conceptual entities: physically flattened as columns on the single articles table (db/schema.sql) rather than separate tables.

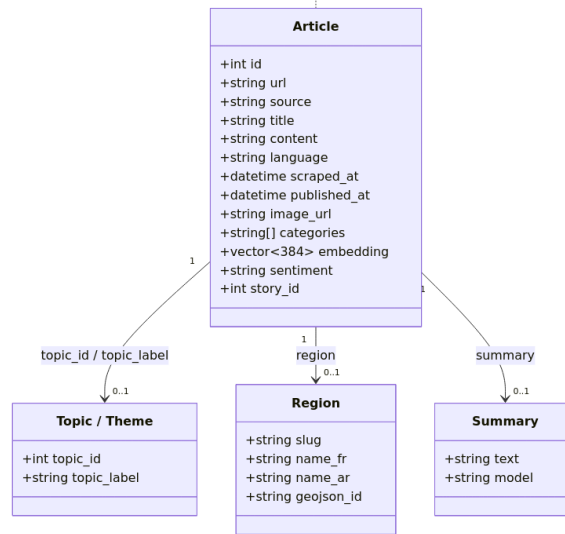


Figure 2.3: Domain model: Article and its conceptual Topic/Theme, Region, and Summary relations.

2.5 Key scenarios

The three sequence diagrams below trace the request flows behind News Insight’s three core features, each spanning the layers introduced in Section 2.2: the background collection pipeline, and the two GenAI-backed, user-triggered flows built on top of it.

2.5.1 Scrape, transform, and load pipeline

Figure 2.4 traces one run of the `news-pipeline` Prefect flow. The `pipeline_task` calls `run_pipeline` (`etl/pipeline.py`), which runs its three steps strictly in sequence: `run_extraction` collects raw `Article` objects from the RSS and HTML scrapers; `transform_articles` computes embeddings, sentiment, and theme/region tags for the whole batch; and `save_articles` upserts the enriched rows into PostgreSQL with `INSERT ... ON CONFLICT (url) DO UPDATE`, so re-running the flow on already-seen URLs is safe and refreshes their enrichment rather than duplicating rows. The pipeline then records aggregate counters (articles saved, sentiment breakdown, embedding model used) to `pipeline_metrics` before returning. If any step raises, Prefect’s task-level retry (two attempts, 60-second delay) re-runs the whole task rather than resuming mid-pipeline.

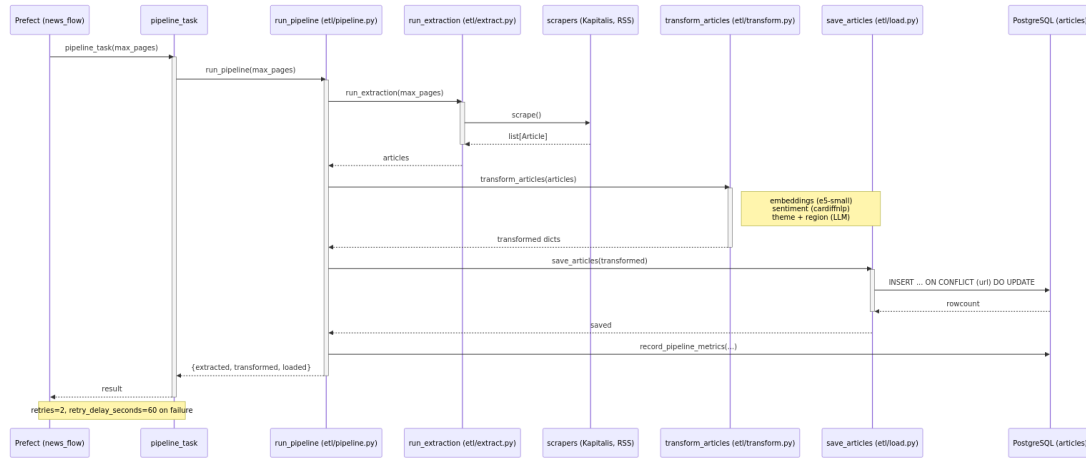


Figure 2.4: Sequence diagram: the scrape / transform / load pipeline, orchestrated by Prefect.

2.5.2 AI-assisted semantic search

Figure 2.5 traces a `GET /search` request (`api/routes/search.py`). The router first checks, cheaply and offline, whether the embedding model’s weights are already cached (`embeddings_available()`); this avoids ever blocking a search request on a multi-minute model download. If the model is available, the query is embedded and the router asks PostgreSQL to order every article by `embedding <=> query_vector` (pgvector’s cosine-distance operator), returning the closest matches with a similarity score. If not, the router falls back to a two-stage keyword-plus-LLM path: it collects candidates from a French full-text match (`tsvector/websearch_to_tsquery`, accent-folded) topped up with recent articles, then asks the LLM to re-rank and filter that candidate pool down to the genuinely relevant ones (`_ai_rank`), so a query like “sport” can still surface an article about a football club acronym that the keyword match alone would miss. Either path returns the same result shape to the frontend.

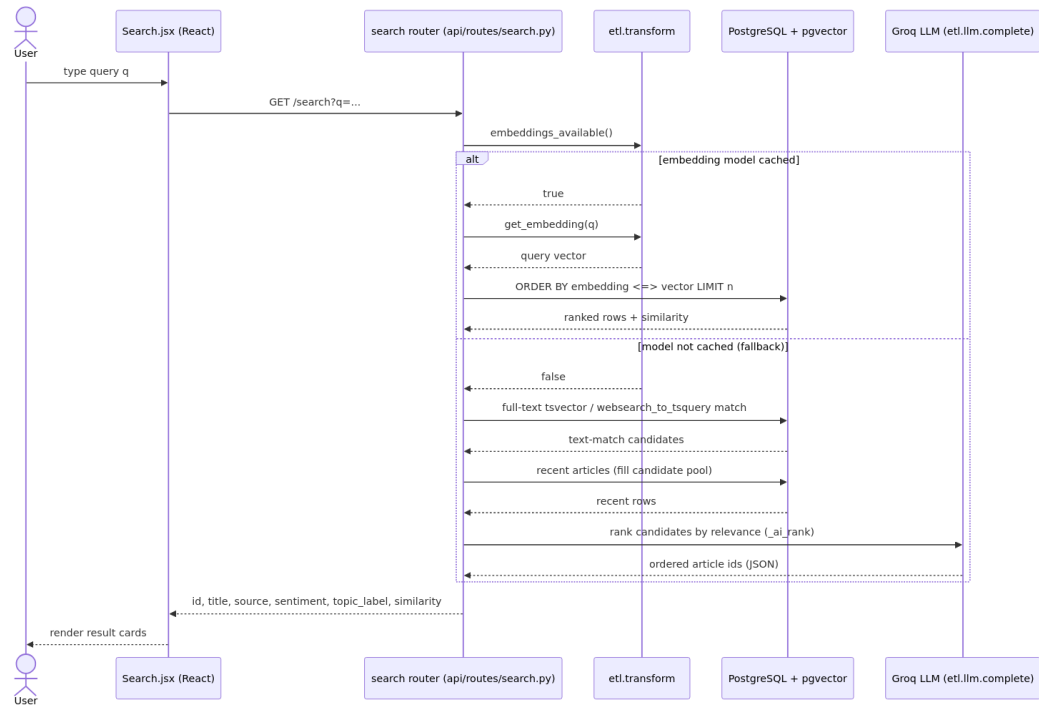


Figure 2.5: Sequence diagram: AI-assisted semantic search, with a keyword + LLM-rank fallback.

2.5.3 Ask-the-news chat (retrieval-augmented generation)

Figure 2.6 traces a `POST /genai/chat` request (`api/routes/genai.py`). It follows the same cached-embedding check as search, retrieving the five closest articles by vector distance when the embedding model is available, or the five highest-ranked by French full-text relevance (falling back to recency) otherwise, always five, so the LLM has real, current articles to ground its answer even on a loose or no-match query. The router then builds a numbered context block from those five articles' titles and excerpts and sends it, together with the user's question, to the LLM with an instruction to answer strictly from the supplied context and cite source numbers. The response returned to the frontend pairs the generated answer with the list of source articles (id, title, url) it was built from, which `ChatWidget.jsx` renders as clickable citations. This retrieval-then-generate structure is what keeps the assistant's answers traceable back to actual News Insight articles rather than the model's unverified prior knowledge.

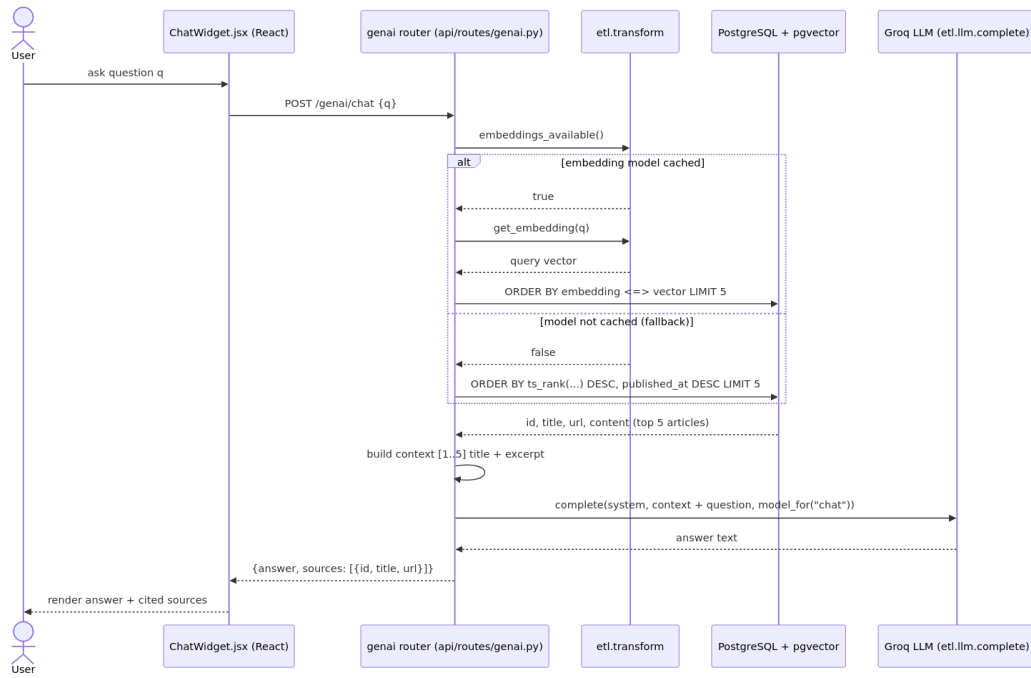


Figure 2.6: Sequence diagram: the ask-the-news RAG chat assistant.

2.6 Conclusion

This chapter moved from the technology choices of Chapter 1 to a concrete system view: a five-stage pipeline from Tunisian news sources to a React dashboard, orchestrated by Prefect and tracked by MLflow, sitting on a single PostgreSQL/pgvector table that keeps every enrichment step a plain column update. The use case, class, and sequence diagrams gave three complementary views of the same system (who uses it, what it stores, and how a request actually flows through it) that the next chapters revisit one layer at a time, starting with the data sources and scraping stage in Chapter 3.

DATA INGESTION AND WEB SCRAPING

Chapter outline

1	Introduction	9
2	High-level architecture	9
3	Actors and use cases	11
4	Domain model	11
5	Key scenarios	12
6	Conclusion	15

3.1 Introduction

Chapter 2 named the two collection mechanisms that feed News Insight (**RSSScraper** and **Kapitalis Scraper**) and placed them at the left edge of the architecture diagram. This chapter opens that box. It explains why a lightweight HTTP-plus-parsing approach was chosen over a browser-automation framework, walks through the shared **BaseScraper** contract both collectors implement, and details each concrete scraper: the HTML listing crawler for Kapitalis and the RSS-based collector used for all four sources. It then covers how the pipeline avoids storing the same article twice and how it degrades gracefully when a single page, feed, or source fails, rather than crashing the whole collection run.

3.2 Scraping technology choice

Tunisian news outlets publish their content as conventional server-rendered HTML pages and, for every source used here, a standard RSS feed. Unlike the JavaScript-heavy, client-rendered catalogs typical of e-commerce sites, neither the listing pages nor the individual articles depend on client-side rendering to expose their text: the full title, body, and metadata are present in the HTML returned by a plain HTTP GET. This removes the main justification for a browser-automation tool such as Playwright or Selenium, which exists to execute JavaScript and wait for asynchronously loaded content, machinery that would add a full browser process, slower page loads, and a heavier dependency footprint for no corresponding benefit here.

News Insight instead uses **requests** for HTTP transport and **BeautifulSoup** (with the **lxml** parser) for HTML parsing, the pairing already listed in Chapter 1’s technology stack (Table 1.1). This combination is synchronous and single-threaded, which is deliberately traded off against the framework’s simplicity: a scraper is a plain Python class with CSS-selector calls, easy to read, debug, and adapt when a source changes its markup, and easy to run unmodified inside a Prefect task without managing a browser’s lifecycle. RSS parsing uses **feedparser**, a purpose-built library for the Atom/RSS XML dialects rather than hand-rolled XML parsing. Where a lighter-weight source can be scraped without a full page fetch, RSS is preferred for exactly that reason: Section 3.5 details why RSS became the primary channel for every source rather than an alternative to it.

3.3 Scraper architecture

Every source-specific scraper is built on a common abstract base, **BaseScraper** (**scrapers/base.py**), which fixes the parts of a scraper that do not vary by source and leaves the parts that do as two abstract methods.

Shared state and helpers. The constructor takes `source`, `base_url`, `language`, and `max_pages` (default 5), and opens a single `requests.Session` configured with a realistic desktop `User-Agent` and an `Accept-Language: fr-FR` header, reused across every request a scraper issues. Two helpers sit on top of that session: `fetch(url)` issues a `GET` with a 10-second timeout, raises on a non-2xx status, and returns a parsed `BeautifulSoup` tree, or `None` if the request fails, logging the error rather than propagating it (Section 3.7); `polite_delay()` sleeps a random 1.5–3.0 seconds, called between every article fetch and after every listing page, so a scraping run does not hammer a source’s server with a tight request loop.

The contract. Two methods are declared abstract and left to each subclass: `get_article_urls(page)`, which returns the list of article URLs found on one listing page, and `parse_article(url)`, which fetches and parses a single article into an `Article` object or returns `None` if parsing fails. `Article` itself (a Pydantic `BaseModel` defined alongside `BaseScraper`) carries `url`, `source`, `title`, `content`, `language`, `categories`, `published_at`, and `image_url`, and enforces at construction time that `title` and `content` are non-empty after stripping whitespace, a validation guard that turns a malformed scrape into an explicit exception rather than a silently empty row.

The template method. `BaseScraper.scrape()` is the one concrete, non-abstract entry point every caller invokes: it loops over pages `1..max_pages`, calls `get_article_urls(page)` on each, stops early if a page returns no URLs, skips any URL already seen earlier in the same run (an in-memory `seen_urls` set, a first layer of deduplication that exists purely to avoid re-parsing the same article twice within one crawl. Section 3.6 covers the persistent, cross-run layer), calls `parse_article(url)` for each new URL, and appends the result when parsing succeeds. `KapitalisScraper` uses this template method as-is; `RSSScraper` overrides `scrape()` entirely, for reasons explained in Section 3.5. Figure 3.1 summarizes both paths from source to database row.

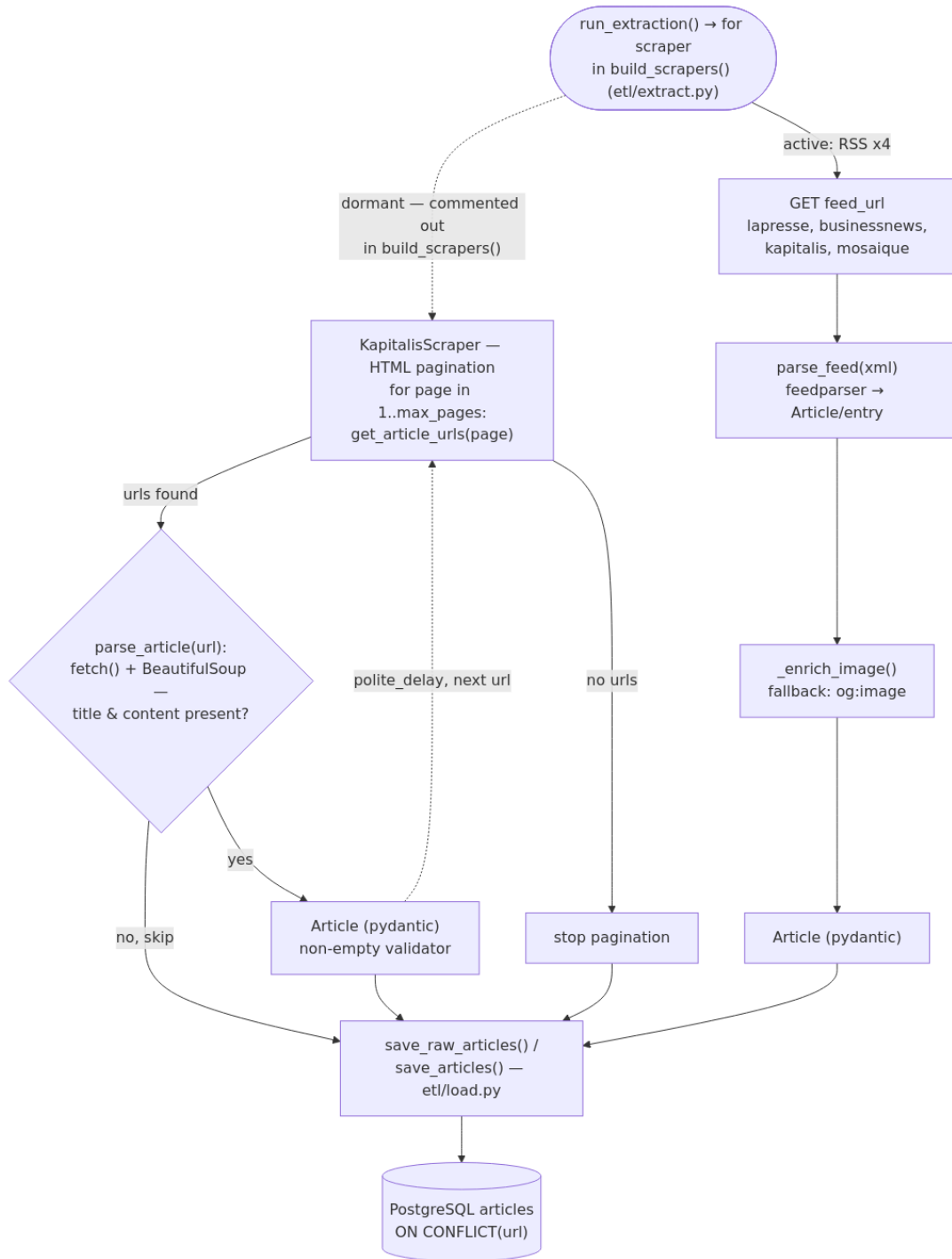


Figure 3.1: Data ingestion flow: the HTML listing path (**KapitalisScraper**) and the RSS path (**RSScraper**) both produce **Article** objects, deduplicated and stored through the same load step.

3.4 Kapitalis scraper

KapitalisScraper (`scrapers/kapitalis.py`) is the one concrete implementation of the full **BaseScraper** template: an HTML listing crawler that pages through Kapitalis’s “à la une” category (`base_url = https://kapitalis.com/tunisie/category/a-la-une`) rather than reading a feed.

`get_article_urls(page)` builds the paginated listing URL `{base_url}/page/{page}/`, fetches it, and selects every anchor matching the CSS path `article.post div h2.entry-title a`, returning

their `href` attributes, one call per page, one article-URL list per call, exactly the contract `BaseScraper.scrape()` expects.

`parse_article(url)` fetches the article page and extracts four fields with dedicated CSS selectors, each written against Kapitalis's WordPress theme markup: the title from `h1.cmsmasters_post_title`; the body from `article div.cmsmasters_post_content.entry-content`, with all tags stripped and inner whitespace collapsed to single spaces (`tag.get_text(separator=" ", strip=True)`), then `" ".join(...split())`; the category list from every article `span.cmsmasters_post_category a`; and the publication date from the `title` attribute of article `abbr.published` (falling back to its visible text). If either title or content is missing after extraction, the method returns `None` instead of constructing a partial `Article`.

This scraper is source-specific by design: every selector above is tied to one theme's markup, so it would need rewriting if Kapitalis changed its template. As Chapter 2 already stated and Section 3.5 confirms from the extraction side, `KapitalisScraper` is kept in the codebase as a deeper-crawl fallback (able to page arbitrarily far back through the category archive, which RSS cannot do) but it is not part of the default extraction run: `build_scrapers` only instantiates it in a commented-out line.

3.5 RSS ingestion

All four active sources (*La Presse*, *Business News*, *Kapitalis*, and *Mosaïque FM*) are collected through `RSSScraper` (`scrapers/rss.py`), configured from the `RSS_FEEDS` list in `etl/extract.py`: four `(source, feed_url, language)` tuples, all tagged `"fr"`. RSS is the primary channel for every source, including Kapitalis, because a feed already delivers a flat, machine-readable list of recent entries with no pagination or listing-page HTML to parse, cheaper to fetch and far less sensitive to a theme redesign than the CSS selectors in Section 3.4.

`RSSScraper` still inherits its constructor, session, and `polite_delay` from `BaseScraper`, but it does not use the inherited `scrape()` template: `get_article_urls` and `parse_article` are both stubbed out (returning `[]` and `None`) because a feed is not paginated the way an HTML listing is, and `scrape()` itself is overridden with a single feed fetch (`max_pages` is fixed to 1 in the constructor). The overridden `scrape()` issues one `GET` on `feed_url`, hands the response body to `parse_feed`, and then calls `_enrich_image` on every resulting article before returning the list.

`parse_feed` runs `feedparser.parse` on the feed body and maps every entry through `_entry_to_article`, the function that turns one feed entry into an `Article`. Its content selection is the most source-specific piece of logic in the RSS path: some feeds (*Mosaïque FM* among them) put only a short caption in the entry's `content` field and the fuller teaser text in `summary`, or the reverse, so

`_entry_to_article` strips HTML tags from both with `BeautifulSoup.get_text` and keeps whichever yields more text (Listing 3.1). Categories come from each entry's `tags` list; `published_at` falls back from `published` to `updated`; and `image_url` is read from the entry's `media_content` field, or, failing that, from any link whose type starts with `image`. An entry missing a link, title, or content is dropped before an `Article` is even constructed; if construction still raises (the same empty-field validator described in Section 3.3), the exception is caught and the entry is skipped with a logged warning rather than aborting the whole feed.

```
content_raw = entry["content"][0].get("value", "") if entry.get("content") else ""
summary_raw = entry.get("summary", "")
content_txt = " ".join(BeautifulSoup(content_raw, "lxml").get_text(" ").split())
summary_txt = " ".join(BeautifulSoup(summary_raw, "lxml").get_text(" ").split())
content = content_txt if len(content_txt) >= len(summary_txt) else summary_txt
```

Listing 3.1: `_entry_to_article`, picking the longer of `content`/`summary` (`scrapers/rss.py`).

Not every feed entry carries an image, so `_enrich_image` runs a best-effort fallback: if the `Article` already has an `image_url` it is left untouched, otherwise the article's own page is fetched (an 8-second timeout, separate from the main feed fetch) and `fetch_og_image` looks for an Open Graph `<meta property="og:image">` tag in its HTML. Any failure here (network error, missing tag, malformed HTML) is caught and only logged at debug level, since a missing thumbnail is a cosmetic gap, not a reason to drop an otherwise valid article.

3.6 Deduplication and raw storage

Deduplication happens in two layers, one in memory and one durable. Within a single `BaseScraper.scrape()` run, the `seen_urls` set (Section 3.3) prevents the same URL from being parsed twice if it appears on more than one listing page, relevant to `KapitalisScraper`, not to `RSSScraper`, whose feed already lists each entry once. That in-memory set is discarded at the end of the run and offers no protection across runs; the durable guarantee comes from the database schema (`db/schema.sql`): `url` is declared `TEXT UNIQUE NOT NULL` on the `articles` table, so a second insert of an already-seen URL is rejected by PostgreSQL itself, independently of what any scraper does or does not remember.

Two load paths write to that table, matching the two run modes introduced in Chapter 2. The lightweight, ML-free path (`scripts/scrape.py`, calling `save_raw_articles` in `etl/load.py`) is meant to persist freshly scraped articles quickly, with no embedding or sentiment fields yet computed; it inserts with `ON CONFLICT (url) DO NOTHING`, so a URL already in the table is left exactly as it is.

A cheap re-run of the scraper never overwrites or duplicates a row it already has. The full pipeline path (`etl/pipeline.py`, calling `save_articles`) is the one already traced in Chapter 2’s sequence diagram (Figure 2.4): it inserts the same way but with `ON CONFLICT (url) DO UPDATE`, refreshing the enrichment columns (`topic_id`, `sentiment`, `embedding`, and so on) computed for that run. Listing 3.2 shows the raw-insert side of that contrast.

```
query = """
    INSERT INTO articles (url, source, title, content, language,
                          published_at, image_url, categories)
    VALUES %s
    ON CONFLICT (url) DO NOTHING
    """
```

Listing 3.2: raw-article upsert, `save_raw_articles` (`etl/load.py`): URL uniqueness is enforced by the database, not by application logic.

Because the constraint lives in the schema rather than in scraper code, both load paths, and any future one, get the same guarantee for free: no scraper needs to query the database before inserting to know whether a URL is new.

3.7 Error handling

Failures are expected at every layer of the ingestion pipeline (a single page can 404, a feed can be temporarily unreachable, a template change can leave a selector matching nothing) and the pipeline is built to lose the smallest possible unit of work on each kind of failure rather than aborting the run.

At the lowest level, `BaseScraper.fetch` catches `requests.RequestException` around every HTTP call, logs the error, and returns `None` instead of raising; every caller (`get_article_urls`, `parse_article`) already treats a missing soup as “nothing found on this page/article” and moves on, so one bad page does not stop the crawl. `KapitalisScraper.parse_article` adds a content-level guard on top: if the title or content selector matches nothing (for instance because the page uses a different template than expected), it returns `None` rather than constructing an `Article` with a blank field. The `Article` model itself provides a second, independent guard for the RSS path through its non-empty-field validator, which `_entry_to_article` wraps in a `try/except` so one malformed feed entry produces a logged warning instead of stopping `parse_feed` from processing the rest of the feed.

At the source level, `run_extraction` (`etl/extract.py`) wraps each scraper’s `scrape()` call in its own `try/except`, logs the exception, and moves on to the next scraper in `RSS_FEEDS`, so a completely

unreachable feed (DNS failure, timeout, malformed XML) costs that one source for that run, not the whole extraction. At the orchestration level, Chapter 2 already described the outermost safety net: Prefect retries the whole pipeline task twice with a 60-second delay if it raises at all, which mainly protects against transient conditions (a momentary network outage, a source under maintenance) that a single retry is likely to clear. Finally, `polite_delay`'s randomized pause between requests is itself a form of error avoidance rather than error recovery: keeping the request rate low against each source reduces the chance of ever triggering a rate limit or a block in the first place.

3.8 Conclusion

This chapter opened the sources box of Chapter 2's architecture diagram: a common `BaseScraper` contract shared by an HTML listing crawler (`KapitalisScraper`, dormant by default) and the RSS-based collector (`RSSScraper`) that actively feeds all four Tunisian sources, converging on the same validated `Article` shape and the same URL-uniqueness guarantee enforced by the database rather than by scraper code. Layered error handling (per-request, per-article, per-source, and, at the orchestration level, per-run) keeps a single bad page or unreachable feed from taking down an entire collection cycle. The next chapter follows these `Article` records into PostgreSQL and details the storage layer they land in.

STORAGE LAYER AND DATABASE DESIGN

Chapter outline

1	Introduction	17
2	Scraping technology choice	17
3	Scraper architecture	17
4	Kapitalis scraper	19
5	RSS ingestion	20
6	Deduplication and raw storage	21
7	Error handling	22
8	Conclusion	23

4.1 Introduction

Chapter 2’s class diagram (Figure 2.3) already sketched the conceptual model behind News Insight’s storage: a single **Article** entity carrying source metadata and pipeline-computed enrichment, with **Topic/Theme**, **Region**, and **Summary** flattened onto it as nullable columns rather than separate tables, plus two operational tables, **entities** and **pipeline_metrics**, sitting outside that conceptual model. This chapter opens the database itself: why PostgreSQL with the **pgvector** extension was chosen as the single engine for both relational metadata and dense vector embeddings, the full column-and-index layout of **articles** and its two auxiliary tables, how vector similarity search is actually executed against the **embedding** column, and how the schema grew from its original shape to today’s through two small, additive migrations. Every table, column, and index described below is read directly from `db/schema.sql` and the two files under `db/migrations/`.

4.2 Database technology choice

News Insight needs to store two different kinds of data per article: structured metadata (source, title, timestamps, sentiment label, topic, region) that benefits from ordinary relational filtering and sorting, and a 384-dimension dense vector per article (Section 4.4) needed for semantic similarity search. A common architecture for the second half is a dedicated vector database (Pinecone, Weaviate, Qdrant, Milvus) running alongside the relational store, with the application responsible for keeping the two in sync: every article would be written twice, once to each system, and a query that needs both a similarity rank and a relational filter (for instance, “the most similar articles that are also recent”) would have to fan out to two systems and merge the results in application code.

News Insight instead uses PostgreSQL with the **pgvector** extension, already listed as the storage row of Chapter 1’s technology table (Table 1.1). **pgvector** adds a **vector** column type and a family of distance operators directly inside Postgres, so the embedding is simply one more column on the **articles** table rather than a record in a second system. The extension is declared at the top of `db/schema.sql`, alongside **unaccent** (used by the accent-insensitive French full-text fallback in `api/routes/search.py`):

```
CREATE EXTENSION IF NOT EXISTS vector;  
CREATE EXTENSION IF NOT EXISTS unaccent;
```

This single-engine choice pays off directly in the codebase: `etl/load.py`’s `get_connection()` is the one connection helper used both by the pipeline’s writer (`save_articles`) and, imported unchanged, by the API’s reader (`api/routes/search.py`, `api/routes/genai.py`): no separate client,

no second set of credentials, and a similarity query can sit in the same `SELECT` as an ordinary `WHERE embedding IS NOT NULL` clause (Section 4.4). The corpus itself is a further argument for this choice: a few thousand articles collected periodically from four Tunisian sources (Chapter 3) is well within reach of pgvector’s exact nearest-neighbor scan, without needing a purpose-built approximate nearest-neighbor (ANN) engine sized for corpora orders of magnitude larger. Finally, Postgres is already the one stateful service in the project’s Docker Compose stack (Chapter 9); enabling pgvector costs nothing beyond an extension flag in the same container, with no additional service to deploy, monitor, or back up.

4.3 Schema design

Figure 4.1 shows the full storage layer as an entity-relationship diagram: the `articles` table at its center, the `entities` table related to it through a foreign key, and the standalone `pipeline_metrics` table.

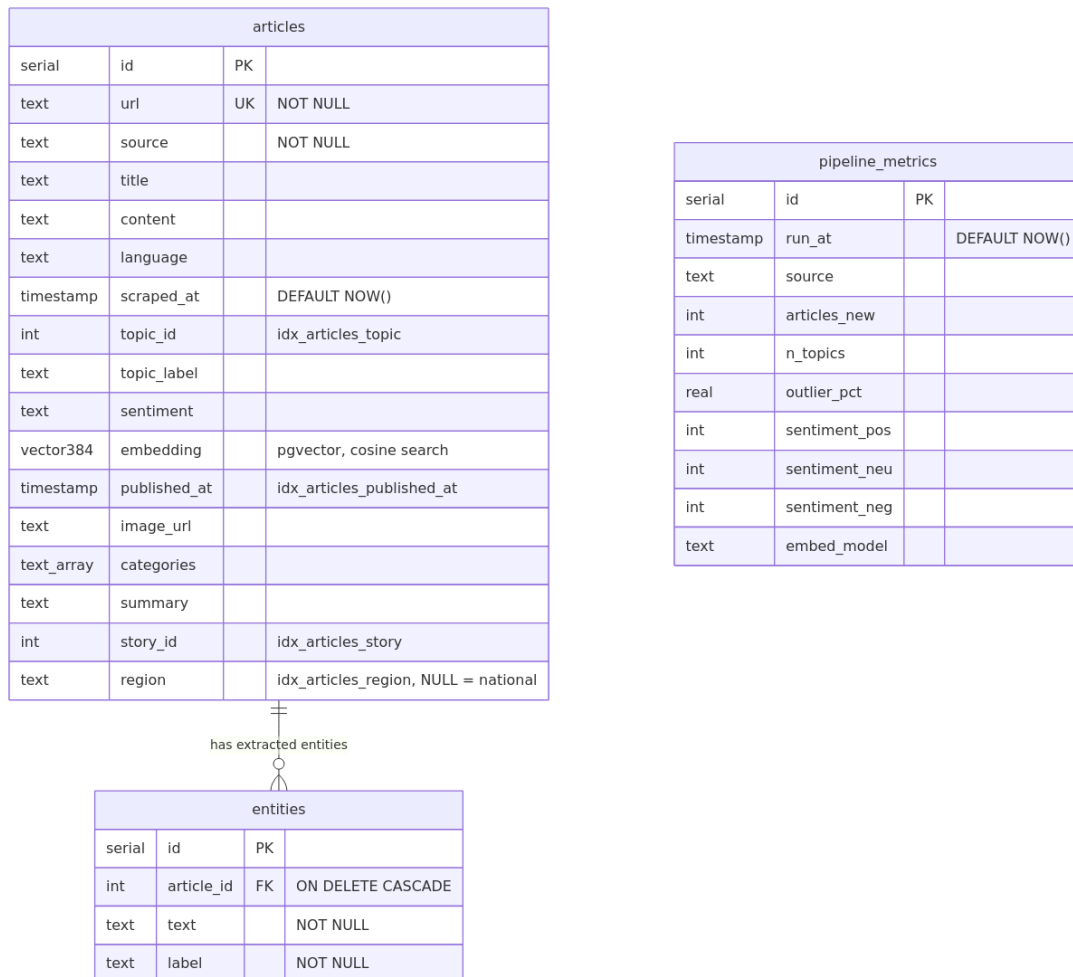


Figure 4.1: Storage layer schema: `articles` (central table, with its pgvector embedding column and indexes), `entities` (one row per extracted named entity, foreign key to `articles`), and `pipeline_metrics` (one row per pipeline run).

`articles` is the table every other part of the system reads from and writes to. Its core columns, read off `db/schema.sql`, are: `id` (SERIAL PRIMARY KEY); `url` (TEXT UNIQUE NOT NULL, the durable deduplication guarantee already described in Chapter 3); `source` and `language`; `title` and `content`; `scraped_at` (TIMESTAMP DEFAULT NOW()) and `published_at` (TIMESTAMP, parsed from the source); `image_url` and `categories` (TEXT[], a native Postgres array rather than a separate join table, since a source’s category list is small and never queried by individual category); `summary` (the on-demand, cached LLM summary from Chapter 2); `story_id` (INTEGER, reserved for a future cross-source story-clustering feature, not yet populated by any pipeline step); `region` (TEXT, added by migration 002, Section 4.5); and the three ML-enrichment columns written by `transform_articles` (Chapter 5): `topic_id`, `topic_label`, `sentiment`, and `embedding vector(384)` (Section 4.4). Listing 4.1 shows the table declaration itself.

```
CREATE TABLE IF NOT EXISTS articles (  
    id          SERIAL PRIMARY KEY,  
    url         TEXT UNIQUE NOT NULL,  
    source      TEXT NOT NULL,  
    title       TEXT,  
    content     TEXT,  
    language    TEXT,  
    scraped_at  TIMESTAMP DEFAULT NOW(),  
    topic_id    INTEGER,  
    topic_label TEXT,  
    sentiment   TEXT,  
    embedding   vector(384),  
    published_at TIMESTAMP,  
    image_url   TEXT,  
    categories  TEXT[],  
    summary     TEXT,  
    story_id    INTEGER,  
    region      TEXT  
);
```

Listing 4.1: the `articles` table (`db/schema.sql`): one row per article, enrichment columns nullable until the pipeline fills them in.

Six B-tree indexes back the queries the dashboard and API actually run: `idx_articles_-topic` on `topic_id` (topic dashboard), `idx_articles_source` on `source` (per-source browsing), `idx_articles_scraped_at` and `idx_articles_published_at` (recency ordering, e.g. the `_recent` fallback in `api/routes/search.py`), `idx_articles_story` on `story_id`, and `idx_articles_region` on `region` (the governorate map dashboard).

No GIN index backs full-text search: the French full-text fallback (Chapter 2, Figure 2.5) computes its `tsvector` inline, at query time, rather than against a stored generated column, so a text-search query runs as a sequential scan rather than an index lookup, an acceptable simplicity trade-off at the corpus’s current size, and the same generated-column-plus-GIN pattern common in Postgres full-text search remains available later if the corpus grows enough for it to matter.

Two further tables sit outside this central table. `entities` (`id`, `article_id` referencing `articles(id)` ON DELETE CASCADE, `text`, `label`) is provisioned for named-entity output, one row per extracted entity per article, with the cascade delete keeping it consistent if an article row is ever removed, but as of this schema no pipeline step inserts into it; it is forward-provisioned schema for a feature not yet wired into `transform_articles`, kept here because Figure 4.1 already documents it as part of the storage layer. `pipeline_metrics` (`id`, `run_at`, `source`, `articles_new`, `n_topics`, `outlier_pct`, `sentiment_pos/neu/neg`, `embed_model`), by contrast, is actively written: `etl/metrics.py`’s `record_pipeline_metrics` inserts one row at the end of every `run_pipeline` call (Chapter 2, Figure 2.4). It exists to answer “what happened in run N” for monitoring rather than to serve article reads, so its use is covered where it is exercised operationally, in Chapter 7.

4.4 Vector storage with pgvector

The `embedding` column’s dimension, 384, is fixed by the embedding model, `intfloat/multilingual-e5-small` (`EMBED_MODEL_NAME` in `etl/transform.py`), a multilingual sentence-transformers model chosen so the same encoder handles French and Arabic source text without a separate model per language. `model.encode(..., normalize_embeddings=True)` is called with `normalize_embeddings=True`, so every stored vector is unit-length, which is what makes pgvector’s cosine-distance operator directly meaningful on the raw column.

The e5 model family distinguishes what is being embedded through a text prefix: `transform_articles` (`etl/transform.py`) prepends `"passage: "` to every article’s title and content and batch-encodes the whole run in one `model.encode(...)` call at ingestion time. The same prefix is also exposed as a single-text helper, `get_passage_embedding`, used by the project’s tests. `get_embedding` instead prefixes with `"query: "` and is what `api/routes/search.py` calls on an incoming search

string. This asymmetric prefixing matches how the model was trained and is what makes a stored passage vector and a live query vector comparable in the same embedding space.

At query time, `api/routes/search.py`'s `semantic_search` first checks `embeddings_available()`, a cheap, offline check for whether the model weights are already in the local Hugging Face cache, avoiding a multi-minute download blocking a search request (already described in Chapter 2). When the model is available, the query is embedded with the `"query: "` prefix, formatted as a pgvector literal, and passed into the query shown in Listing 4.2: `<=>` is pgvector's cosine-distance operator (0 for identical direction, larger for more dissimilar vectors), used both to order the results and, subtracted from 1, to report a similarity score back to the frontend. The same `embedding <=> ...` pattern, gated behind the same `embeddings_available()` check, is reused by `api/routes/genai.py` to retrieve the five closest articles as grounding context for the “ask the news” chat assistant (Chapter 2): one column and one operator serving two features.

```
SELECT id, url, source, title, sentiment, topic_label,  
       1 - (embedding <=> %s::vector) AS similarity  
FROM articles  
WHERE embedding IS NOT NULL  
ORDER BY embedding <=> %s::vector  
LIMIT %s
```

Listing 4.2: pgvector cosine-distance search (`semantic_search`, `api/routes/search.py`): `<=>` ranks articles by embedding similarity to the query vector.

The `WHERE embedding IS NOT NULL` guard filters out rows the pipeline has not yet enriched: raw articles saved by the lightweight, ML-free load path (`save_raw_articles`, Chapter 3) have no embedding until the full pipeline's transform step runs over them. `db/schema.sql` declares no ANN index (no `ivfflat` or `hnsw` index) on `embedding`, so every similarity query above runs pgvector's exact, brute-force distance scan over all non-null embeddings rather than an approximate lookup. Combined with the scale argument from Section 4.2, this is a deliberate choice rather than an oversight: an ANN index trades a small amount of recall for query speed at large scale, a trade not worth making yet at a few thousand rows, and one that can be added later with a single `CREATE INDEX ... USING ivfflat` statement without changing any application code, since it only changes how Postgres executes the same `<=>` query, not the query itself.

4.5 Migrations

The schema did not arrive in its current shape in one step. `db/migrations/001_news_hub.sql` is an additive migration meant to be run against an already-deployed database: every statement is either `ADD COLUMN IF NOT EXISTS` or `CREATE ... IF NOT EXISTS`, so re-running it against a database that already has these changes is a no-op rather than an error. It adds the `published_at`, `image_url`, `categories`, `summary`, and `story_id` columns to `articles` (the column set behind the article-preview and summary features described in Chapter 2) together with their two indexes, and it creates the `entities` and `pipeline_metrics` tables (Section 4.3).

`db/migrations/002_region.sql` is narrower, adding exactly the `region` column and its index for the Tunisia governorate map feature:

```
ALTER TABLE articles ADD COLUMN IF NOT EXISTS region TEXT;
CREATE INDEX IF NOT EXISTS idx_articles_region ON articles(region);
```

Listing 4.3: region-tagging migration (`db/migrations/002_region.sql`): a nullable column, `NULL` meaning an article the region tagger could not confidently place in one of Tunisia’s governorates.

`db/schema.sql` itself already contains the union of the base schema and both migrations: every column above appears directly in its `CREATE TABLE IF NOT EXISTS articles` block, so it is not a hand-maintained snapshot that could drift from the migration history but the same additive statements folded into one script. A fresh environment bootstraps straight from `schema.sql`; an existing deployment instead applies the two migrations, in order, to converge on the same final shape without ever dropping data (Chapter 9 covers how this runs inside the Docker Compose stack). Every `ALTER` across both migrations is `ADD COLUMN`, never a drop or a rename, so the two layer cleanly on the original schema without needing a migration framework such as Alembic or Flyway, a deliberate simplicity choice for a project at this size and history, at the cost of no automatic down-migration and no separate schema-version tracking table.

4.6 Conclusion

This chapter opened the database behind Chapter 2’s class diagram: a single PostgreSQL instance, extended with pgvector, storing both an article’s structured metadata and its 384-dimension multilingual embedding in one `articles` table, indexed for the access patterns the dashboard and API actually exercise, alongside a forward-provisioned `entities` table and an actively written `pipeline_metrics` table. pgvector’s `<=>` cosine-distance operator is the one primitive both semantic search and the

chat assistant’s retrieval step build on, run as an exact scan deliberately rather than through an approximate index, at a corpus size where that trade is not yet needed. Two small, additive migrations grew the schema to its current shape without a migration framework and without ever touching data already collected. The next chapter follows the **embedding**, **sentiment**, **topic_id**, and **region** columns introduced here into the machine learning pipeline that actually computes them.

MACHINE LEARNING AND NLP PIPELINE

Chapter outline

1	Introduction	25
2	Database technology choice	25
3	Schema design	26
4	Vector storage with pgvector	28
5	Migrations	30
6	Conclusion	30

5.1 Introduction

Chapter 4 closed by following the `embedding`, `sentiment`, `topic_id/topic_label`, and `region` columns of the `articles` table back to “the machine learning pipeline that actually computes them.” This chapter is that pipeline. It reads six files end to end: `etl/transform.py`, `etl/cluster.py`, `etl/themes.py`, `etl/labels.py`, `etl/regions.py`, and `etl/metrics.py`, and, together with the orchestration entry point `etl/pipeline.py`, explains exactly which model computes which column, with which parameters, and in which order. Three model families do the heavy lifting: a multilingual sentence-transformer for semantic embeddings, a fine-tuned XLM-RoBERTa classifier for sentiment, and BERTopic for unsupervised topic discovery, each wrapped by lighter, code-level heuristics (an LLM theme classifier, an LLM topic-labeller, and a gazetteer-first region tagger) that turn raw model output into the columns Chapter 4 described. Figure 5.1 is the map referred to throughout the chapter: the per-article transform stage on top, the periodic clustering stage below it, and the PostgreSQL `articles` table both stages write to.

5.2 Pipeline overview

`run_pipeline` (`etl/pipeline.py`) is the single entry point Chapter 2’s sequence diagram (Figure 2.4) already traced: four steps, run strictly in sequence, with no step started before the previous one returns. **Extract** (`run_extraction`, Chapter 3) collects raw `Article` objects from the RSS and HTML scrapers; if it returns nothing, the function logs a warning and returns immediately with a zeroed result dictionary, so nothing downstream ever runs on an empty batch. **Transform** (`transform_articles`, Section 5.3) is where embeddings, sentiment, theme, and region are all computed for the whole batch. **Load** (`save_articles`, Chapters 3–4) upserts the enriched rows with `ON CONFLICT (url) DO UPDATE`. **Metrics** (`record_pipeline_metrics`, Section 5.7) writes one summary row and is wrapped in its own `try/except` so that a metrics-write failure is logged and swallowed rather than failing an otherwise successful run. The module-level constant `EMBED_MODEL = "intfloat/multilingual-e5-small"` is defined here purely to be recorded on that metrics row; the model itself is loaded from a separate, identically-valued constant, `EMBED_MODEL_NAME`, inside `etl/transform.py`: the two are kept in sync by hand rather than imported from one shared definition.

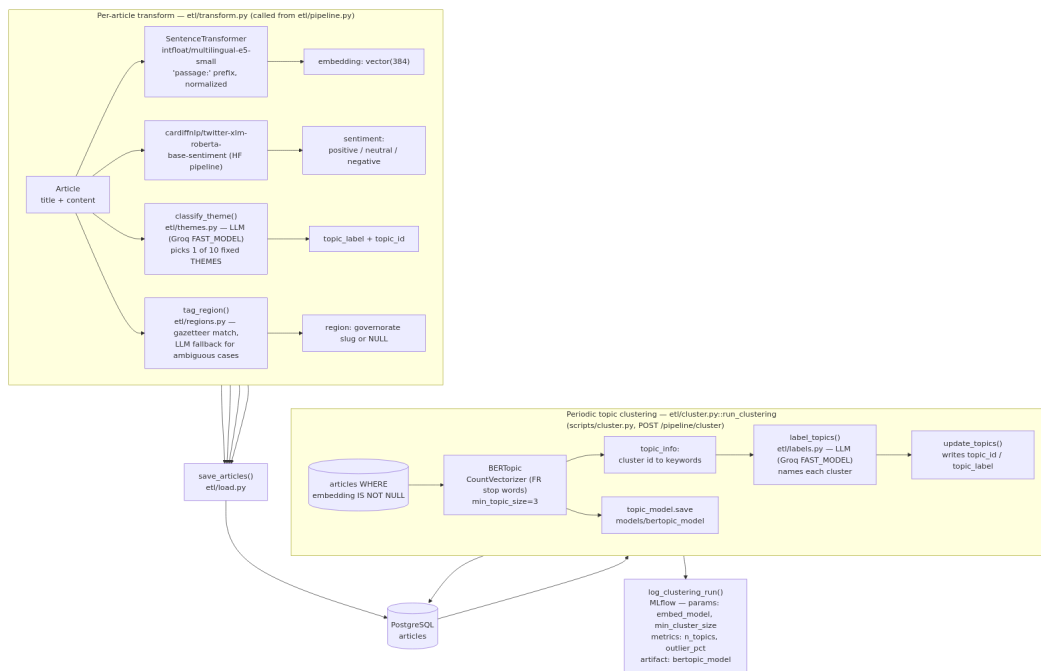


Figure 5.1: The ML/NLP pipeline: per-article transform (embeddings, sentiment, theme, region) feeding `articles`, and the separate, periodic BERTopic clustering stage that reads embeddings back out and overwrites `topic_id/topic_label`.

`run_pipeline` itself is reachable three ways: directly (`python -m etl.pipeline`), wrapped in a Prefect task with two retries and a 60-second delay (`flows/news_flow.py`, Chapter 7), or triggered on demand as a FastAPI background task (`POST /pipeline/trigger`, `api/routes/pipeline.py`). Notably, BERTopic clustering (Section 5.5) is *not* one of its four steps: it needs a batch of already-embedded articles and enough of them to fit a topic model, so it runs as a separate stage, invoked by an operator through `scripts/cluster.py` (`make cluster`, `MIN_CLUSTER=3` by default) or `POST /pipeline/cluster`. A third, narrower entry point exists purely for local development: `scripts/transform.py` (`make transform`, `BATCH_SIZE=16` by default) re-embeds and re-scores only the articles still missing an embedding (`fetch_untransformed_articles`, `etl/load.py`), writing embedding and sentiment back one row at a time through `update_article_ml_fields`. It deliberately skips theme and region entirely, unlike `transform_articles`, which computes all four in one batch.

5.3 Text embeddings

`transform_articles` opens by building one text per article ("`passage:` " followed by the title and content, matching the e5 prefixing convention already introduced from the storage side in Chapter 4) and encoding the whole batch in a single call to `model.encode`, rather than one call per article (Listing 5.1). The embedding model, `intfloat/multilingual-e5-small` (`EMBED_MODEL_NAME`), is loaded once through a lazy singleton, `_get_embedding_model`: a module-level `_embedding_model` variable is populated on first use and reused afterwards, so a script that only needs, say, region tagging

never pays the cost (a few hundred megabytes of weights) of loading it, and the FastAPI process does not load it at startup either. `embeddings_available()`, already described in Chapter 4 from the search side, is the same offline cache check reused here conceptually: the model is downloaded from the Hugging Face Hub on first real use if not already cached locally.

```
texts = [
    f"passage: {(a.title or '')} {(a.content or '')}".strip()
    for a in articles
]

embeddings = model.encode(
    texts,
    normalize_embeddings=True,
    batch_size=16,
    show_progress_bar=True,
)
```

Listing 5.1: batched e5 passage encoding (`transform_articles`, `etl/transform.py`): one `model.encode` call for the whole run.

`normalize_embeddings=True` makes every stored vector unit-length, the property Chapter 4 relied on to make pgvector's `<=>` cosine-distance operator directly meaningful. `batch_size=16` here is a value hard-coded inside `transform_articles` itself, distinct from the Makefile's configurable `BATCH_SIZE` variable, which instead controls the batch size passed to `scripts/transform.py`'s own, separate `model.encode` call (Section 5.2). Each resulting vector is 384-dimensional, fixed by the model architecture and matching the `vector(384)` column declared in `db/schema.sql`. The asymmetric prefix is what makes this a *passage* embedding rather than a *query* embedding: `get_embedding`, the function `api/routes/search.py` calls on an incoming search string, prefixes with `"query: "` instead, so a stored article vector and a live search-query vector are comparable in the same trained embedding space even though they went through different code paths.

5.4 Sentiment analysis

Sentiment is computed by `cardiffnlp/twitter-xlm-roberta-base-sentiment`, a fine-tuned XLM-RoBERTa model, loaded lazily the same way as the embedding model through `_get_sentiment_pipeline`: a Hugging Face `transformers.pipeline("text-classification", ...)`, with `truncation=True` and `max_length=512` tokens, and `device=0` if `torch.cuda.is_available()`

`else -1` so the same code runs on GPU where available and falls back to CPU otherwise. `transform_articles` calls the batched helper, `get_sentiments`, on the plain `"{title} {content}"` text. Sentiment does not use the `e5` prefix, which is specific to the embedding step. Each text is first truncated to its first 1000 characters (a cheap pre-truncation ahead of the model's own 512-token limit), and the whole list is scored in one `pipe(..., batch_size=16)` call.

The model's predicted label is lower-cased and looked up in `SENTIMENT_MAP`, a dictionary mapping `"positive"`, `"neutral"`, and `"negative"` to themselves, with `.get(label, "neutral")` so that any label the map does not recognize (not just I/O or model errors) silently resolves to `"neutral"` instead of propagating. This is deliberate defensive coding rather than a translation between two different label vocabularies: `get_sentiments` additionally wraps the whole batched call in a `try/except` that defaults *every* text in the batch to `"neutral"` on any failure, so a broken sentiment model degrades the run's sentiment quality rather than aborting it, the same never-break-the-pipeline philosophy `run_pipeline` applies to its own metrics write (Section 5.2). A singular counterpart, `get_sentiment`, exists in the same module with identical truncation and mapping logic but is not called from anywhere else in the codebase as of this pipeline.

5.5 Topic modeling with BERTopic

BERTopic clustering (`run_clustering`, `etl/cluster.py`) is the data-driven counterpart to the per-article theme classifier of Section 5.6: instead of asking an LLM to place one article into one of ten fixed categories, it looks at the embeddings of *every* article with one already assigned and discovers topics as clusters in that vector space. `load_articles_for_clustering` pulls `id`, `title`, `content`, and `embedding` for every row where `embedding` IS NOT NULL; if fewer than ten articles qualify, the function logs a warning and returns `{"status": "skipped", "reason": "not enough articles"}` without fitting anything (Listing 5.2): BERTopic's own clustering step needs a minimum amount of data to find stable clusters at all.

```
if len(ids) < 10:
    logger.warning(
        f"Not enough articles: {len(ids)}, "
        "need at least 10"
    )
    return {"status": "skipped",
           "reason": "not enough articles"}
```

Listing 5.2: the minimum-corpus guard (`run_clustering`, `etl/cluster.py`): clustering never runs

on fewer than ten embedded articles.

Article text is vectorized for keyword extraction by a `CountVectorizer` with `min_df=1` (a term needs to appear in only one document to be kept, appropriate at the project's corpus size), French stop words from `spacy.lang.fr.stop_words`, and `ngram_range=(1, 2)` so single words and two-word phrases are both candidate topic keywords. `BERTopic` is then constructed with `vectorizer_model=vectorizer`, `min_topic_size=min_cluster_size` (a `run_clustering` parameter defaulting to 3, and exposed as the Makefile's `MIN_CLUSTER` variable through `scripts/cluster.py`'s `--min-cluster-size` flag), `language="french"`, and `calculate_probabilities=False`. No custom `umap_model` or `hdbscan_model` is passed in, so dimensionality reduction and density-based clustering both run with `BERTopic`'s own internal defaults, applied on top of the e5 embeddings already computed by `transform_articles` and passed straight in through `fit_transform(texts, embeddings=embeddings_array)`. `BERTopic` never recomputes its own embeddings here, it clusters the ones already stored in `articles.embedding`.

`get_topic_info()` then returns one row per discovered cluster, each with an integer topic id and a raw, auto-generated `Name` (`BERTopic`'s own keyword-concatenated string), excluding topic -1, the reserved id for articles that did not fit any dense cluster (`HDBSCAN`'s outlier bucket). These raw names, together with each topic's top keywords from `topic_model.get_topic(tid)`, are handed to `label_topics` (Section 5.6) to be turned into short, human-readable labels before anything is written back; the whole `label_topics` call is wrapped in a `try/except` that falls back to the raw `BERTopic` names if label generation fails outright. The fitted model is persisted to `models/bertopic_model` (`topic_model.save(...)`), and `update_topics` overwrites `topic_id/topic_label` for every clustered article, including the outliers, which get `topic_id=-1` and whatever label `topic_labels.get(-1, "unknown")` resolves to, since -1 was excluded from the labelled dictionary. This is a genuine overwrite of the theme values `transform_articles` assigned earlier: an article's `topic_id/topic_label` pair reflects whichever ran most recently, LLM theming at ingestion time or `BERTopic` clustering afterwards. Finally, `outlier_pct` is computed as the fraction of articles assigned topic -1, and `log_clustering_run` records `embed_model` and `min_cluster_size` as MLflow parameters and `n_topics/outlier_pct` as MLflow metrics under the `news-clustering` experiment, with the saved model directory logged as an artifact. The operational side of this run is covered in Chapter 7.

5.6 Enrichment: themes, labels and regions

Three lighter mechanisms fill in the remaining enrichment columns, each combining a small deterministic or LLM step with a hard-coded fallback.

Themes. `classify_theme` (`etl/themes.py`) is what `transform_articles` calls for every article, immediately, well before BERTopic clustering ever runs. It is the theme mechanism referred to as “the LLM theme classifier” in Chapter 2. It sends the article’s title and the first 400 characters of its content to a fast LLM with a French prompt naming exactly ten fixed categories (`THEMES`: Politique, Économie, Sport, Société, International, Culture, Sécurité, Santé, Environnement, Technologie) and asks for one category name back, capped at `max_tokens=8`. The reply is matched against the fixed list by `_match`, which normalizes accents and case and accepts either an exact match or a substring containment; any parsing failure, LLM error, or unmatched reply defaults to “Société”. `theme_id` then converts the chosen name to the integer stored in `topic_id` by taking its index in `THEMES`, and, distinct from BERTopic’s genuine `-1` outlier sentinel, falls back to `len(THEMES) - 1` (the index of “Technologie”, the last entry) for any name it does not recognize, rather than to a sentinel meaning “no theme.” The LLM call itself is imported locally inside `classify_theme`, the same lazy-loading pattern used for the embedding and sentiment singletons in Section 5.3. Provider and model-selection details behind this and the next LLM call are covered in Chapter 6.

Labels. `label_topics` (`etl/labels.py`) is what turns a BERTopic cluster’s raw, keyword-concatenated name into something readable: for each topic id it joins up to eight of the cluster’s top keywords and asks a fast LLM for a two-to-four-word French title with no closing punctuation, capped at `max_tokens=24`. Each per-topic call is individually guarded: a failure for one topic falls back to the literal string `f"Topic {tid}"` rather than aborting the whole labelling pass, which is itself wrapped again by `run_clustering`’s outer fallback to the raw BERTopic names (Section 5.5): two independent layers of degradation between a fully LLM-labelled cluster set and the model’s own auto-generated names.

Regions. `etl/regions.py` holds a gazetteer of Tunisia’s 24 governorates (`GOVERNORATES`): for each, a slug, French and Arabic names, an ISO 3166-2 `geojson_id` for the map dashboard, and a list of aliases covering name variants and major cities or delegations. `normalize` strips Latin accents (Unicode NFD decomposition, dropping combining marks) and lower-cases text while leaving Arabic letters intact, and `ALIAS_INDEX` flattens every alias across all 24 governorates, sorted longest-first so a multi-word place name is matched before a shorter substring of it. `tag_region_gazetteer` scores each governorate by counting word-boundary alias matches, weighting a title hit three times a body hit (`_score_regions`), and returns the strict single winner: `None` if no alias matched at all, or if two or more governorates tie for the highest score, rather than guessing. A second tier, `tag_region_llm`, asks an LLM to pick one of the 24 slugs or “national” when the gazetteer alone is ambiguous, validating the reply against the known slug set before trusting it. `tag_region(title, content, use_llm=True)` chains the two: gazetteer first, LLM only if `use_llm` is true and the gazetteer found

nothing. The function’s own default is `use_llm=True`, but `transform_articles` explicitly calls it with `use_llm=False`, so in the pipeline that actually runs, region tagging is gazetteer-only; an article the gazetteer cannot place unambiguously is simply left with `region = NULL`, shown on the frontend map as the “National / non-localisé” bucket (`region_label(None)`), and `tag_region_llm` is fully implemented and tested but never exercised by this call site.

5.7 Pipeline metrics

`record_pipeline_metrics` (`etl/metrics.py`) does one thing: insert one row into `pipeline_metrics` (Chapter 4) with whatever dictionary it is given. Its only caller is `run_pipeline`, at the very end of every full run, and the values it passes reveal where the two stages of this chapter do and do not talk to each other. `source` is always the fixed string `"all"`: the column exists for a per-source breakdown the current code never produces, one row is written per full pipeline run regardless of how many distinct sources it scraped. `articles_new` is the row count `save_articles` returned. `sentiment_pos/sentiment_neu/sentiment_neg` are counted directly off the in-memory `transformed` list by comparing each article’s `sentiment` field. `embed_model` is the `EMBED_MODEL` constant introduced in Section 5.2. `n_topics` and `outlier_pct`, by contrast, are always `None` on this path: `run_pipeline` never runs BERTopic clustering (Section 5.5), so it has no topic count or outlier rate to report. `run_clustering` does compute both, but sends them to MLflow’s `news-clustering` experiment via `log_clustering_run` instead: as of this pipeline, no code path ever inserts a `pipeline_metrics` row with `n_topics` or `outlier_pct` populated, and the two columns Chapter 4 documented in the schema remain schema-only for clustering, tracked instead through MLflow. The operational use of both destinations is picked up in Chapter 7.

5.8 Conclusion

This chapter followed every enrichment column Chapter 4 introduced back to the model call or heuristic that fills it in: `intfloat/multilingual-e5-small` for 384-dimension, prefix-asymmetric, batch-encoded embeddings; `cardiffnlp/twitter-xlm-roberta-base-sentiment` for batched, fail-safe-to-neutral sentiment; a ten-category LLM classifier for the theme every article gets at ingestion time, overwritten on demand by BERTopic (French-stop-word vectorized, `min_topic_size=3` by default, guarded against fewer than ten articles, its raw cluster names relabelled by a second LLM pass, its `-1` bucket a genuine “no cluster” outlier distinct from the theme classifier’s forced ten-way choice); and a gazetteer-first, LLM-fallback-disabled-by-default tagger over Tunisia’s 24 governorates for region. A single `pipeline_metrics` row closes out every full run, though its

topic-related columns stay empty on that path, populated conceptually only by the separate MLflow tracking of clustering runs. The next chapter turns to the LLM layer these theme, label, and region calls all quietly depend on: the provider-agnostic client behind `etl/llm.py`, and the GenAI features (summaries and the “ask the news” chat assistant) built on top of it.

GENAI FEATURES

Chapter outline

1	Introduction	33
2	Pipeline overview	33
3	Text embeddings	34
4	Sentiment analysis	35
5	Topic modeling with BERTopic	36
6	Enrichment: themes, labels and regions	37
7	Pipeline metrics	39
8	Conclusion	39

6.1 Introduction

Chapter 5 closed by handing off “the LLM layer these theme, label, and region calls all quietly depend on” to this chapter: `etl/llm.py`. That module is the one place in the codebase that knows how to talk to a large language model at all; every file that needs an LLM call, `etl/themes.py`’s `classify_theme`, `etl/labels.py`’s `label_topics`, and `etl/regions.py`’s `tag_region_llm` (all three read in Chapter 5), plus the two files read in full here, `api/routes/genai.py` and `api/routes/search.py`, imports from it rather than building its own client. This chapter covers what Chapter 5 deliberately deferred: how `etl/llm.py` abstracts over an LLM provider, how the two GenAI features exposed directly to a News Insight reader, on-demand article summaries and the “ask the news” retrieval-augmented chat assistant, are built on top of it, and how AI-assisted semantic search borrows the same provider to re-rank results when the embedding model is unavailable. Chapter 2’s sequence diagrams (Figures 2.5 and 2.6) already traced these last two flows at the architecture level; this chapter goes one layer deeper, into the actual prompts, model choices, and two real production bugs (one in article summaries, one in search relevance) that shaped the code as it stands today.

6.2 LLM provider abstraction

`etl/llm.py` is a thin, provider-agnostic wrapper around any OpenAI-compatible chat-completions API: three environment variables are enough to swap the model behind every GenAI feature in the project without touching a single call site. `LLM_API_KEY` is the provider’s API key. `LLM_BASE_URL` is the OpenAI-compatible base URL, defaulting to `DEFAULT_BASE_URL = "https://api.groq.com/openai/v1"`, Groq, chosen as the default because, per the module’s own docstring, its free tier needs no card and has the highest requests-per-day allowance among the alternatives it documents: Google Gemini (<https://generativelanguage.googleapis.com/v1beta/openai/>, “best French, long context”) and OpenRouter (<https://openrouter.ai/api/v1>, an aggregator exposing free `:free`-suffixed models at a lower rate limit). `_client()` (Listing 6.1) builds a single `openai.OpenAI` client lazily, on first use, and caches it in a module-level `_client_singleton`, the same lazy-singleton pattern Chapter 5 described for the embedding and sentiment models, avoiding an unnecessary network client (and an import of the `openai` package) in any process path that never actually calls an LLM.

```
def _client():
    global _client_singleton
    if _client_singleton is None:
        from openai import OpenAI
```

```
_client_singleton = OpenAI(
    base_url=os.getenv("LLM_BASE_URL", DEFAULT_BASE_URL),
    api_key=os.getenv("LLM_API_KEY"),
    default_headers={"X-Title": "Tunisia News Hub"},
)

return _client_singleton
```

Listing 6.1: the provider client, built entirely from environment variables (`_client`, `etl/llm.py`): changing `LLM_BASE_URL` alone repoints every GenAI feature at a different OpenAI-compatible provider.

`complete(user, system=None, max_tokens=512, model=None)` is the one function every caller in the codebase uses to talk to the model: it assembles an optional system message and the user message into the `messages` list the chat-completions API expects, calls `_client().chat.completions.create(...)`, and returns `resp.choices[0].message.content`, stripped of surrounding whitespace. No caller ever touches the raw HTTP response. If `model` is not passed explicitly, `complete` falls back to `os.getenv("LLM_MODEL", DEFAULT_MODEL)`, where `DEFAULT_MODEL = "llama-3.3-70b-versatile"`. Distinct from that per-call fallback, `model_for(feature, default)` is a second, explicit resolution helper most call sites use instead: it checks `LLM_MODEL_<FEATURE>` (upper-cased: `LLM_MODEL_SUMMARY`, `LLM_MODEL_LABELS`, `LLM_MODEL_CHAT`), falls back to the blanket `LLM_MODEL`, and only then to the `default` argument the call site itself passes in. This per-feature indirection exists specifically to keep Groq's free-tier rate limits comfortable: a fast, cheap model, `FAST_MODEL = "llama-3.1-8b-instant"`, is the code-level default for high-volume, lower-stakes work (summaries, topic labels), while the heavier `DEFAULT_MODEL` is the code-level default for chat, where per-call answer quality matters more and call volume is naturally lower: one call per user question rather than one per article. Both are Groq free-tier models; the split is a cost/throughput trade-off within a single provider by default, and every one of `LLM_MODEL_SUMMARY`, `LLM_MODEL_LABELS`, and `LLM_MODEL_CHAT` can still be overridden independently without code changes.

6.3 Article summaries

`GET /genai/summary/{article_id}` (`summary`, `api/routes/genai.py`) is the simplest GenAI feature and the only one with a persistent cache: it first selects `summary`, `title`, `content` for the article and returns the cached value immediately if `summary` is already non-NULL, so a given article is only ever sent to the LLM once, no matter how many times a reader opens it. On a cache miss, it calls `complete` with `model=model_for("summary", FAST_MODEL)` (`llama-3.1-8b-instant` by default), a system prompt

instructing the model to summarize in French in one or two clear sentences, and a user message pairing the title with the first 2000 characters of the content; the response is capped at `max_tokens=160` and written back with `UPDATE articles SET summary=%s WHERE id=%s` before being returned. On the frontend, `ArticleModal.jsx`'s `ArticleSummary` component fetches this endpoint once per opened article, shows “Résumé...” while waiting, and renders nothing at all (no error banner) if the request fails or the summary comes back empty.

The system prompt above is stricter than it looks, and for a documented reason: pull request #9, *fix(genai): summaries refuse on thin-content articles*, found that some summaries came back as the model asking for the article instead of summarizing it, for instance, for “ESS: La FTF ne ‘cèdera’ pas Anis Boujelbène,” a Mosaïque FM item. The root cause traced back one layer further, into `scrapers/rss.py`: Mosaïque FM’s RSS feed puts a short caption (“Anis Boujelbène,” about fifteen characters) in the `<content>` element and the real, roughly 149-character teaser in `<summary>`, and the scraper unconditionally preferred `<content>`, so every Mosaïque FM article was stored with almost no body text. Handed a near-empty article, the 8B model reasonably asked for more information rather than fabricating a summary. The fix has two parts: `scrapers/rss.py` now keeps whichever of the parsed `<content>/<summary>` text is longer, so future scrapes store the real teaser; and, since existing summaries could not be re-scraped, the prompt in `api/routes/genai.py` was hardened to state explicitly that it must summarize *whatever* title and text it is given, “même s’ils sont courts,” and must “ne demande jamais d’autres informations”, with the user message itself changed from a bare title-plus-body concatenation to an explicit “**Titre : ... Texte : ...**” pairing that substitutes the placeholder “(aucun corps fourni)” when the stripped content is empty, so the model is never handed a blank line to react to. Because summaries are cached, the fix alone did not repair already-stored refusals: the pull request notes a one-off backfill that lengthened 27 existing Mosaïque FM rows and cleared their cached refusal text, a reminder that a persistent LLM cache is also a persistent record of whatever prompt produced it.

6.4 LLM topic labels and themes

Chapter 5 already covered the mechanics of the two other LLM-backed enrichment steps in detail: `classify_theme` (Section 5.6), which assigns one of ten fixed French categories to every article at ingestion time, and `label_topics` (also Section 5.6), which turns a BERTopic cluster’s raw keyword list into a short human-readable title. Both are built on exactly the provider layer described above, imported from the same `etl/llm.py`: `classify_theme` and `tag_region_llm` call `complete(..., model=FAST_MODEL)` directly, while `label_topics` instead resolves its model through

`model_for("labels", FAST_MODEL)`, so a topic-labelling model swap can be made independently via `LLM_MODEL_LABELS`, while theme classification and the LLM region fallback have no such per-feature override and move only with the blanket `LLM_MODEL` or the shared `FAST_MODEL` constant. Nothing about their prompts or fallback behaviour differs from Chapter 5’s account; they are included here only to make the point that *every* LLM call in the system, whether it runs silently during ingestion or on demand from a user request, goes through the one abstraction in Section 6.2.

6.5 AI semantic search

Chapter 4 already covered `GET /search/`’s primary path: when `embeddings_available()` is true, results are ordered directly by pgvector’s `<=>` cosine distance and no LLM is involved at all. The LLM only enters when the embedding model’s weights are not cached locally. In that case, `semantic_search` (`api/routes/search.py`) builds a candidate pool of up to `CAND_LIMIT = 40` rows: `_text_matches` first collects French full-text matches (`websearch_to_tsquery`, accent-folded via `unaccent`, ranked by `ts_rank`), and `_recent` tops that pool up with the most recently published articles not already in it, so the LLM always has a reasonably sized, de-duplicated set of candidates to choose from even for a query the text index barely matches. `_ai_rank(q, rows, limit)` then sends the candidate `id: titre` listing to the LLM with a strict French prompt: return a JSON array of *only* the genuinely relevant article ids, most relevant first, understanding sense rather than surface keywords (its own example: “sport” should match football, a match, a club, or a team acronym like “EST”/“ESS”), and return an empty array rather than pad the list if nothing truly fits. The reply is pulled out with a small regular expression, `re.search(r"\[[\d,\s]*\]", out)`, parsed as JSON, and cast to a list of integers; any exception anywhere in that chain (a malformed reply, a provider error) is caught and treated as “the LLM found nothing usable,” returning an empty list rather than raising. Ranked ids are then mapped back to full rows and, if the LLM’s list came back empty, the endpoint falls back to `_text_matches`’ own text-rank order rather than the full candidate pool, so an unmatched query still returns genuine text matches, or nothing, never unrelated recency-padded articles.

That last fallback is itself the fix for a real production bug: commit *fix(search): AI search returned irrelevant results (padded to the limit)*, already named among Chapter 1’s branch history as `fix/ai-search-precision`. The symptom was that a query for “sport” returned twenty results, several of them clearly unrelated: articles about Trump, Iran, and a heatwave (*canicule*). The root cause was two-fold: `_ai_rank` at the time called the weaker `FAST_MODEL` (8B) to rank candidates, which either over-returned ids beyond what was truly relevant or occasionally produced output the regex/JSON parse could not extract at all; and, whenever that ranking failed or came back short, the

code's own fallback padded the result out to `candidates[:limit]`, which included the recency-padding rows `_recent` had added, i.e. whatever was scraped most recently regardless of topic. The fix changed two things: `_ai_rank` now calls the stronger `DEFAULT_MODEL` (70B, imported directly from `etl.llm` rather than resolved through `model_for`; search ranking has no `LLM_MODEL_SEARCH` override, only the blanket `LLM_MODEL`), with the stricter, sense-aware, empty-array-allowed prompt described above; and the failure fallback was narrowed from the whole candidate pool to text matches only, described in the code as returning “real matches or nothing, not random news.” The commit message records the fix's verification directly against the reported symptom: *sport* returned 7 football/World-Cup results, *iran* returned 5 Middle-East results, *canicule* returned 5 weather results, *économie* returned 4 economy results, with “no off-topic bleed.” Chapter 10 shows the frontend view built on this path (Figure 10.1), after the same fix.

6.6 Ask-the-news chat assistant

`POST /genai/chat` (`chat`, `api/routes/genai.py`) accepts a single field, `q`, and implements a small retrieval-augmented-generation loop. Retrieval follows the same cached-embedding check as search: if `embeddings_available()`, the question is embedded (with the “`query:` ” prefix described in Chapter 4) and the five nearest articles are fetched by `ORDER BY embedding <=> %s::vector LIMIT 5`; otherwise, the same French full-text machinery used elsewhere ranks by `ts_rank` and falls back to `COALESCE(published_at, scraped_at)` recency, again `LIMIT 5`. The comment in the code is explicit about why this path always returns five rows rather than nothing on a weak match: chat needs *some* grounding context to answer at all, unlike search, which must never show the reader an article it cannot justify. The five retrieved rows (`id`, `title`, `url`, `content`) are then folded into one numbered context block (Listing 6.2): each article becomes “[`n`] `title`\n`content[:800]`”, joined by blank lines, giving the model up to 800 characters of body text per source.

```
context = "\n\n".join(
    f"[{i+1}] {r[1]}\n{r[3] or ''[:800]}"
    for i, r in enumerate(rows)
)
```

Listing 6.2: building the numbered RAG context (chat, api/routes/genai.py): five retrieved articles become five bracketed, citable sources.

That context, together with the question, is sent to `complete` with a system prompt instructing the model to answer strictly from the supplied articles, cite their source numbers, and say so explicitly

if the answer is not in them; the model is resolved through `model_for("chat", DEFAULT_MODEL)` (llama-3.3-70b-versatile by default) and capped at `max_tokens=512`. The endpoint returns `{"answer": ..., "sources": [{"id", "title", "url"}]}`, with `sources` always listing all five retrieved articles regardless of which bracket numbers the generated answer actually cites, leaving it to the frontend to present them as the full evidence set rather than a filtered citation list. `ChatWidget.jsx` renders this as a floating assistant: a round action button toggles a docked panel offering three canned French suggestions on an empty conversation (“*Que se passe-t-il en Tunisie cette semaine ?*” among them), posts each question to `/genai/chat`, appends the returned answer as an assistant bubble with its sources rendered as a bulleted list of clickable external links, and, on a failed request, appends a generic French error bubble instead of the answer, catching the failure entirely on the client. Figure 6.1 shows the assistant open with an answered question and its cited sources.



Figure 6.1: The “ask the news” chat assistant (`ChatWidget.jsx`): a RAG answer over five retrieved articles, with clickable source citations.

6.7 Cost and reliability considerations

Every design choice in this chapter traces back to running LLM features on a free tier without a card on file. The fast/strong model split (Section 6.2) is the main lever: `FAST_MODEL` absorbs the highest-volume traffic (one `classify_theme` call per ingested article, one `label_topics` call per BERTopic cluster, one summary call per distinct article ever opened) while `DEFAULT_MODEL` is reserved for the lower-volume, higher-stakes calls: one per chat question, and, since the precision fix, one per AI-search request that reaches the ranking step. Summaries add a second lever on top, a persistent cache in `articles.summary`: the same article is never summarized twice, regardless of

how many readers open it, which chat has no equivalent for since every question is, by construction, distinct. Because `LLM_BASE_URL/LLM_MODEL` are the only inputs `etl/llm.py` needs, the whole system can also move off Groq entirely (to Gemini for better French and a longer context window, or to an OpenRouter `:free` model for a different rate-limit profile) without a code change, exactly as the README documents.

Reliability is handled asymmetrically across the codebase, and not by accident. `_client()` configures no explicit request timeout or retry policy, so every call to `complete` inherits whichever defaults the underlying `openai` Python SDK (and the `httpx` client it wraps) ships with. Chapter 5's three enrichment functions (`classify_theme`, `label_topics`, `tag_region_llm`) and this chapter's `_ai_rank` all wrap their own `complete` call in a `try/except` that degrades to a hard-coded default ("Société," `f"Topic {tid}"`, an unmatched region, an empty ranking list) rather than raising, so a provider outage during a batch pipeline run or a search request lowers output quality without aborting anything. The two live, user-triggered endpoints in `api/routes/genai.py`, `summary` and `chat`, do not follow that pattern: neither wraps its `complete` call in a `try/except`, so a provider error at request time propagates as an unhandled exception and FastAPI turns it into a 500 response. Both frontend call sites already assume this can happen and handle it locally instead of relying on the backend to degrade gracefully: `ArticleModal.jsx`'s `ArticleSummary` catches the failed request and simply renders no summary box at all, and `ChatWidget.jsx`'s `ask` catches it and appends a generic French error bubble to the conversation rather than leaving the user staring at a stalled spinner.

6.8 Conclusion

A single, roughly fifty-line module, `etl/llm.py`, underlies every LLM-backed feature this report has described across two chapters: theme classification and topic labelling at ingestion time (Chapter 5), and article summaries, AI-assisted search ranking, and the ask-the-news chat assistant covered here, all swappable to a different provider through three environment variables, and individually re-modelled through three more. Two real fixes drawn directly from the project's commit history illustrate that this is a system that was actually run and iterated on rather than designed once and left alone: a scraper-plus-prompt fix for articles too short for the model to summarize, and a stronger-model-plus-stricter-fallback fix for search results that drifted into unrelated recent news whenever the ranking step failed. The next chapter turns from these individual features to how the whole pipeline that feeds them is orchestrated and observed in production: Prefect flows and MLflow experiment tracking.

ORCHESTRATION AND EXPERIMENT

TRACKING

Chapter outline

1	Introduction	42
2	LLM provider abstraction	42
3	Article summaries	43
4	LLM topic labels and themes	44
5	AI semantic search	45
6	Ask-the-news chat assistant	46
7	Cost and reliability considerations	47
8	Conclusion	48

7.1 Introduction

Chapter 1 named *reproducible MLOps* as one of News Insight’s four founding objectives (wrapping the pipeline in tooling that makes it observable and repeatable rather than a collection of ad hoc scripts) and listed Prefect and MLflow among the project’s core technology choices (Table 1.1). Chapter 2’s architecture overview already named both services by their exact role (Prefect wrapping the scrape-transform-load sequence in one flow, **news-pipeline**, and MLflow logging every BERTopic run under the **news-clustering** experiment), and its sequence diagram (Figure 2.4) already traced one full flow run down to the task-level retry. Every chapter since has deferred a piece of the operational story to this one: Chapter 3’s error-handling section named Prefect’s task retry as “the outermost safety net” (Section 3.7) without describing how it is configured; Chapter 5 named all three ways **run_pipeline** is reachable, including “wrapped in a Prefect task with two retries and a 60-second delay (**flows/news_flow.py**, Chapter 7)”, and closed both its BERTopic section and its metrics section by pointing the reader here (Sections 5.5 and 5.7); and Chapter 6 ended by handing off exactly “how the whole pipeline that feeds them is orchestrated and observed in production: Prefect flows and MLflow experiment tracking.” This chapter reads that operational layer end to end: the flow and task definitions in **flows/news_flow.py**, the cron deployment in **flows/schedule.py**, the MLflow logging call in **etl/cluster.py**, and the two Docker Compose services and environment variables, **PREFECT_API_URL** and **MLFLOW_TRACKING_URI**, that connect the two.

7.2 Orchestration technology choice

requirements.txt pins **prefect>=3.0.0**; Chapter 1’s technology table already listed the choice at a glance (“Prefect: scheduling and running the scrape / transform / cluster pipeline”) without arguing for it over the field’s other well-known option, Apache Airflow. The two occupy different points on the same trade-off Chapter 3 made when it picked **requests+BeautifulSoup** over a full browser-automation stack: simplicity against the ability to express a bigger problem. Airflow is built around a DAG-of-operators model, aimed at coordinating many discrete, interdependent tasks across a schedule, and it pays for that with its own always-on scheduler process, a metadata database, and tasks authored against its own operator API rather than as plain Python functions. News Insight’s pipeline, by contrast, is the single straight-line sequence Chapter 2 already traced (extract, transform, load, strictly in that order, per Section 5.2), with no branching and no cross-task fan-out to schedule around. Prefect’s **@flow/@task** decorators (Section 7.3) let that existing sequence, **etl.pipeline.run_pipeline**, become a scheduled, retried, and observable unit without being rewritten against a different execution model at all: two decorators on top of code that would

otherwise just be called directly, which is the right amount of orchestration machinery for one linear job run by a small team, rather than the DAG-authoring and infrastructure overhead a many-task, many-dependency system like Airflow is built to justify.

7.3 Flow design

Figure 7.1 is the map this section and the next two follow: the two ways the pipeline starts (a cron-scheduled deployment, Section 7.4, or a direct HTTP trigger, below), the flow/task pair inside `flows/news_flow.py`, the three-step `run_pipeline` call it wraps (already read in full in Chapter 5, Section 5.2), and the two places a run becomes observable afterward: the Prefect UI and the `pipeline_metrics` table (Section 7.6).

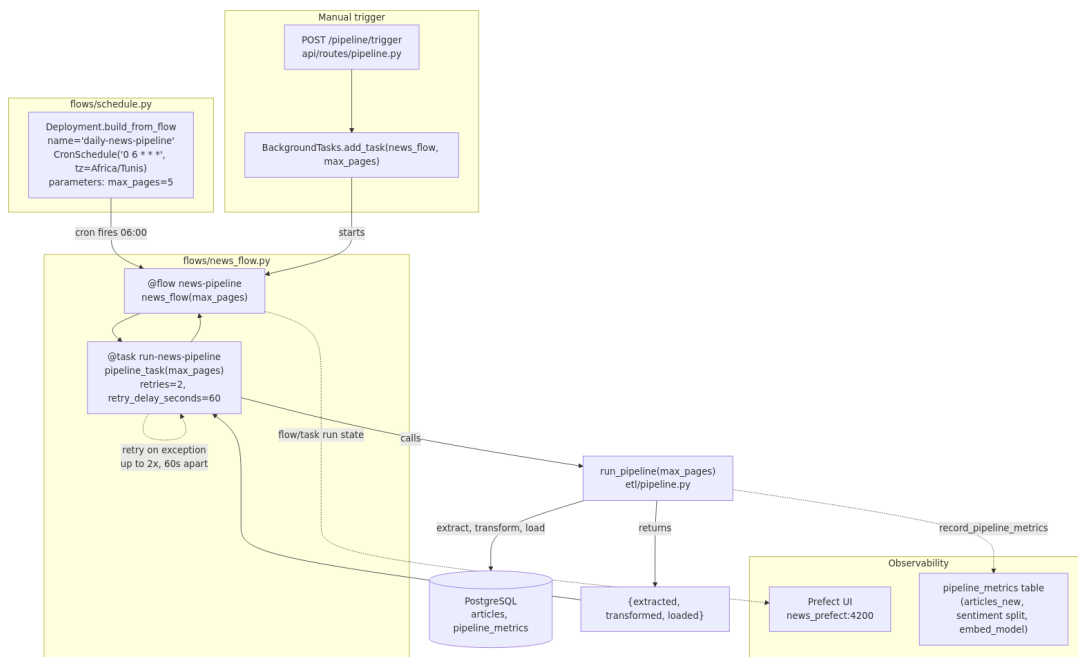


Figure 7.1: Orchestration map: how a run starts (cron deployment or manual HTTP trigger), the `news-pipeline` flow and its single retried task, the `run_pipeline` steps it wraps, and the two observability endpoints (the Prefect UI and the `pipeline_metrics` table) covered later in this chapter.

`flows/news_flow.py` (Listing 7.1) defines exactly one task and one flow. `pipeline_task`, decorated `@task(name="run-news-pipeline", retries=2, retry_delay_seconds=60)`, does nothing but call `run_pipeline(max_pages=max_pages)` and return its result dictionary. The whole three-step `extract/transform/load` sequence is wrapped by a single Prefect task, not decomposed into one task per step. `news_flow`, decorated `@flow(name="news-pipeline", description="Scrape, transform and load Tunisian news articles")`, logs a start message, calls `pipeline_task(max_pages=max_pages)`, logs the returned result, and returns it; its own `max_pages` parameter defaults to 5. Because the task boundary sits around the entire `run_pipeline` call rather than around each

of its three steps, Prefect's retry is coarse-grained by design: if `transform_articles` raises after `run_extraction` already succeeded, the retry re-runs the whole task, extraction included, rather than resuming from the failed step. That repeated work is harmless rather than destructive because of a guarantee from two chapters back (`save_articles`'s `INSERT ... ON CONFLICT (url) DO UPDATE`, Chapter 2), so re-scraping already-seen URLs on a retried attempt refreshes their rows instead of duplicating them.

```
@task(name="run-news-pipeline", retries=2, retry_delay_seconds=60)
def pipeline_task(max_pages: int) -> dict:
    return run_pipeline(max_pages=max_pages)

@flow(
    name="news-pipeline",
    description="Scrape, transform and load Tunisian news articles",
)
def news_flow(max_pages: int = 5):
    logger.info("Starting news pipeline flow")
    result = pipeline_task(max_pages=max_pages)
    logger.info(f"Flow completed: {result}")
    return result
```

Listing 7.1: the flow and its one task (flows/news_flow.py): `run_pipeline`'s three-step sequence (Chapter 5) is wrapped by a single retried task, not split one task per step.

Two entry points can start this flow outside its cron schedule. `POST /pipeline/trigger` (`trigger_pipeline`, `api/routes/pipeline.py`) hands `news_flow` to FastAPI's `BackgroundTasks.add_task`, so the flow runs in-process after the HTTP response has already been sent back to the caller; its own `max_pages` default is 3, smaller than the flow function's own default of 5 and smaller than the daily deployment's parameter (Section 7.4), a lighter default for what is more often a quick manual check than the full daily batch. Because `news_flow` is itself a Prefect `@flow`, calling it this way (directly, from a background task, with no deployment or worker involved) still emits a flow run to whichever server `PREFECT_API_URL` points at, so both the cron-scheduled path and this on-demand path end up as visible runs in the same Prefect UI (Figure 7.2). `POST /pipeline/cluster` (`trigger_clustering`) is the pipeline's other background-task endpoint, but it calls `run_clustering` (`etl/cluster.py`) directly, a plain function, carrying no `@task` or `@flow` decorator at all. Clustering

runs, whether started this way or from `scripts/cluster.py` (`make cluster`), therefore never appear in the Prefect UI; MLflow’s `news-clustering` experiment (Section 7.5) is their only observability surface. The two halves of the pipeline are consequently watched by two different tools, each blind to the other’s runs.

One naming overlap is worth flagging so it does not read as a contradiction later in this chapter: the Makefile’s own `make pipeline` target (`scrape` then `transform` then `cluster`, three standalone scripts run locally, per the README) is a different sequence from the Prefect flow this section describes. The Prefect flow is named `news-pipeline`, but it only ever wraps `run_pipeline` (`extract`, `transform`, `load`) and never runs BERTopic clustering; the two “pipelines” share a name in casual usage but not a code path.

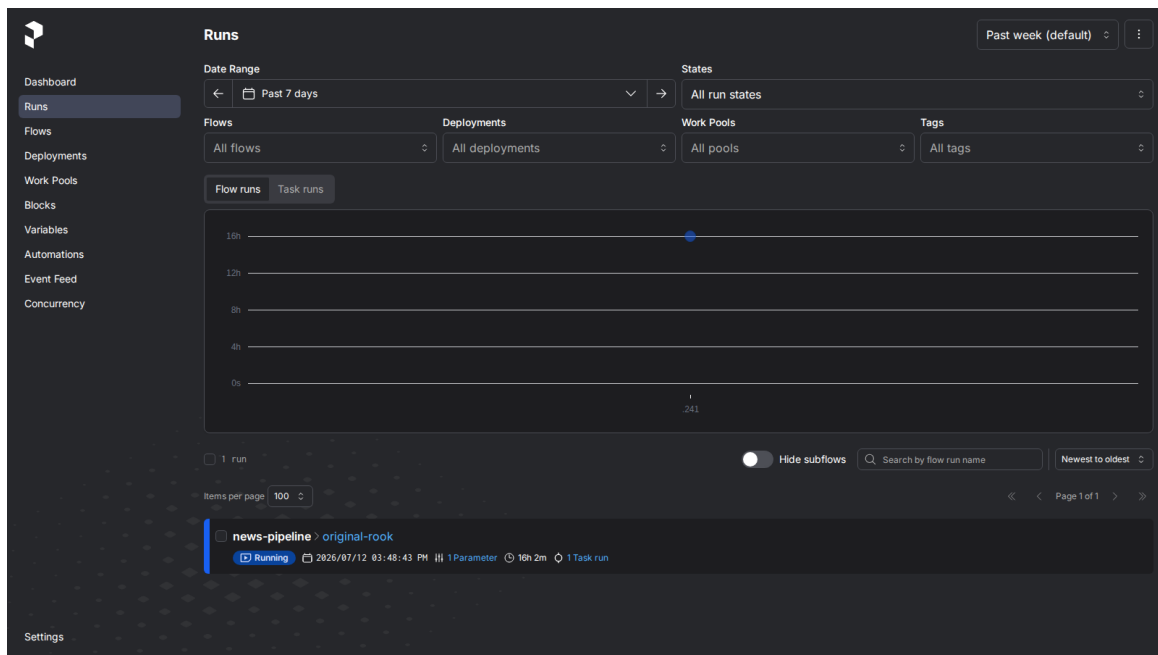


Figure 7.2: Prefect UI’s Runs view, showing a `news-pipeline` flow run in progress, with its single task run and elapsed duration.

7.4 Scheduling

`flows/schedule.py` registers exactly one deployment via `Deployment.build_from_flow:`

```
flow news_flow, name "daily-news-pipeline", schedule CronSchedule(cron="0 6 * * *",
    timezone="Africa/Tunis"), and parameters={"max_pages": 5}, followed by deployment.apply()
```

when the module is run as a script. The cron expression fires once a day at 06:00, in the `Africa/Tunis` timezone specifically rather than whatever timezone the server’s own clock happens to be in (ahead of the Tunisian news day, so a run picks up whatever was published overnight before readers open the dashboard in the morning), and always with the same `max_pages=5` the flow’s own default already uses. Running `python -m flows.schedule` is a registration step, not a long-running process: it

pushes this deployment’s definition to whichever Prefect server `PREFECT_API_URL` names, and only needs re-running when the schedule or its parameters change.

Registering a deployment and actually executing it on schedule are two different things in Prefect 3.x, and only the first of the two is wired up in this project as it stands. `docker-compose.yml`’s `prefect` service runs `prefect server start --host 0.0.0.0` (the API and UI process a deployment is registered *against*), but executing a deployment’s scheduled runs needs a separate worker process polling a work pool for it, and no such worker or agent service is defined anywhere in `docker-compose.yml`. Neither the Makefile, `docker-compose.yml`, nor the README’s run instructions ever invoke `flows/schedule.py` either. A `daily-news-pipeline` deployment, once registered, would appear in the Prefect UI’s deployment list with its cron schedule attached; nothing in the current containerized stack polls and executes it automatically. In today’s setup, the two paths that actually run `news_flow` are the two direct ones already covered: a manual `python -m flows.news_flow` invocation (the module’s own `__main__` block, `max_pages=1`), and the on-demand `POST /pipeline/trigger` endpoint, both calling the flow directly, bypassing the scheduled deployment entirely.

7.5 Experiment tracking with MLflow

`log_clustering_run(n_topics, outlier_pct, min_cluster_size)` (`etl/cluster.py`) is `run_clustering`’s last step (Chapter 5, Section 5.5), called once every `BERTopic` run completes and `update_topics` has already written `topic_id/topic_label` back to the `articles` table. It lazily imports `mlflow` into a module-level placeholder on first call (the same lazy-import discipline Chapter 5 and Chapter 6 already described for the embedding, sentiment, and LLM clients, keeping `import etl.cluster` light for any process that never actually clusters), then points the client at `os.getenv("MLFLOW_TRACKING_URI", "http://localhost:5000")` and calls `mlflow.set_experiment("news-clustering")`, so every clustering run, regardless of which of the entry points in Section 7.3 started it, lands in the same named experiment. Inside `mlflow.start_run()` (Listing 7.2), `log_params` records `embed_model` (hard-coded here as the literal string `"intfloat/multilingual-e5-small"` rather than imported from the `EMBED_MODEL` constant Chapter 5 named in `etl/pipeline.py`, Section 5.2, the same value duplicated rather than shared) and `min_cluster_size`, the caller’s own parameter (3 by default, Section 5.5). `log_metrics` records `n_topics` and `outlier_pct`, computed exactly as Chapter 5 described: the count of non-outlier `BERTopic` clusters, and the fraction of articles assigned topic -1.

The saved model directory is logged last, and not with a single unconditional call:

```
(mlflow.log_artifacts if os.path.isdir("models/bertopic_model") else
```


`mlflow.log_artifact("models/bertopic_model")` picks between MLflow's directory-logging and single-file-logging functions at call time. This is itself the fix for a real bug, recorded in commit *fix: timeline days lower bound + robust mlflow artifact logging*: the original code called `log_artifacts` (plural) unconditionally, but BERTopic's own `.save()` may write a single file rather than a directory depending on its serialization backend, and `log_artifacts` cannot accept a bare file path, so a save that happened to produce one file would fail this last logging step outright. `run_clustering` wraps the entire `log_clustering_run` call in its own `try/except`, logging a warning on failure rather than raising, so (exactly like the asymmetric degrade-instead-of-fail pattern Chapter 6 described for LLM calls) an MLflow outage costs only the tracking record for that run, never the clustering result already committed to `articles.topic_id/topic_label`.

```
mlflow.set_tracking_uri(os.getenv("MLFLOW_TRACKING_URI", "http://localhost:5000"))
mlflow.set_experiment("news-clustering")
with mlflow.start_run():
    mlflow.log_params({"embed_model": "intfloat/multilingual-e5-small",
                      "min_cluster_size": min_cluster_size})
    mlflow.log_metrics({"n_topics": n_topics, "outlier_pct": outlier_pct})
    (mlflow.log_artifacts if os.path.isdir("models/bertopic_model")
     else mlflow.log_artifact)("models/bertopic_model")
```

Listing 7.2: logging one clustering run (`log_clustering_run`, `etl/cluster.py`): parameters, metrics, and the fitted model as an artifact, all under the `news-clustering` experiment.

`MLFLOW_TRACKING_URI`'s code-level default, `http://localhost:5000`, matches the README and `.env.example`'s own documented default for local, non-Docker use. Inside `docker-compose.yml`, the `api` container overrides it to `http://mlflow:5000` (the `mlflow` service's container-internal port, reached by its Compose service name over the internal network), while the same server is reachable from a browser on the host at `http://localhost:5001`, since the `mlflow` service's own port mapping, `"5001:5000"`, exists specifically to avoid a host port already claimed by another local service (the file's own comment names macOS AirPlay as the usual culprit). Figure 7.3 is one such run, viewed through that host-mapped UI.

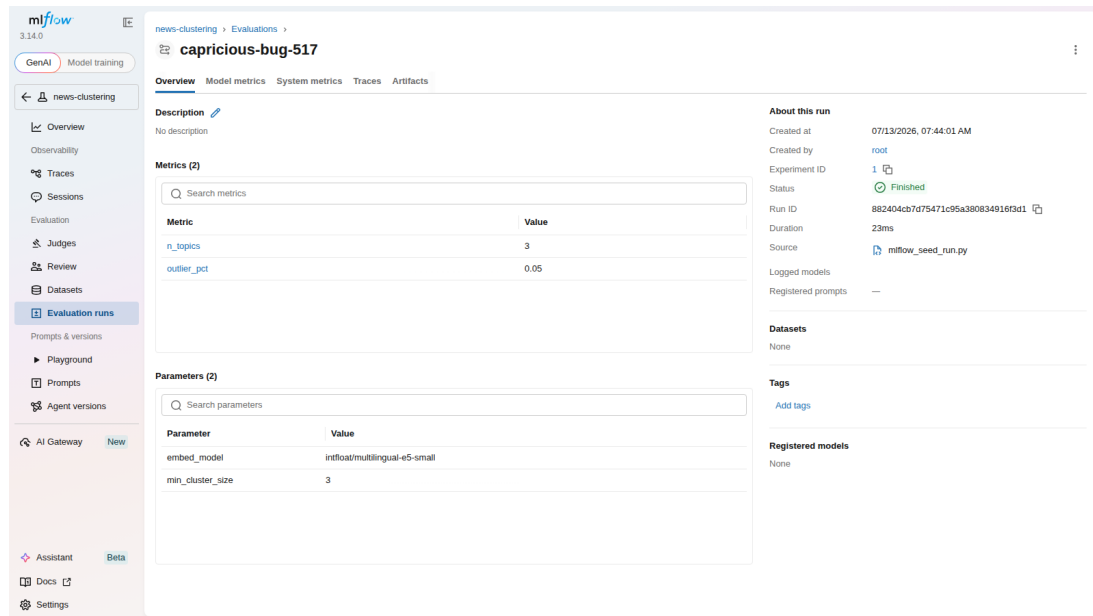


Figure 7.3: MLflow tracking UI showing a logged clustering run: parameters, metrics, and the fitted model artifact.

Chapter 5 already noted, from the `pipeline_metrics` side, that `n_topics` and `outlier_pct` stay `None` on every row `run_pipeline` writes, because clustering is a separate stage that never runs inside `run_pipeline` itself (Section 5.7). This section closes that loop from the MLflow side: those two values are not simply missing from the system, they are tracked in a different place entirely, one row per BERTopic run under the `news-clustering` experiment rather than one row per `pipeline_metrics` insert. “Was the last pipeline run healthy” and “was the last clustering run healthy” are, as a result, two separate questions answered by two separate stores (PostgreSQL for the first, MLflow for the second) that never populate the same row.

7.6 Monitoring and reliability

`docker-compose.yml`’s health configuration mirrors the failure-handling asymmetry the code itself already encodes. The `api` service’s `depends_on` only requires `postgres` and `prefect` to report `service_healthy` (each backed by its own `healthcheck` block, `pg_isready` for Postgres and a small `urllib`-based request against `/api/health` for Prefect), while `mlflow` carries no `healthcheck` at all and is absent from `api`’s `depends_on` list entirely. A slow-starting or crashed MLflow container therefore never blocks the API from starting, which matches exactly what the code already guarantees at the call-site level (Section 7.5): MLflow is wired in as a best-effort side channel, not a dependency the rest of the system waits on or fails without.

That same fail-loud/fail-quiet split runs through `run_pipeline` itself (`etl/pipeline.py`, Chapter 5). Its three core steps (`run_extraction`, `transform_articles`, `save_articles`) carry

no `try/except` of their own at that level; an exception in any of them propagates all the way up through `pipeline_task` to Prefect, which is exactly what makes the task-level retry (Section 7.3) the mechanism that handles it, two attempts, 60 seconds apart, rather than a swallowed error. Only the very last step, `record_pipeline_metrics`, is wrapped in its own `try/except` (the pipeline’s own comment reads “never break the pipeline on a write failure”), so a database hiccup while writing the `pipeline_metrics` row costs only that row, not the scrape/transform/load work already committed ahead of it. `run_clustering` repeats the same pattern twice over: the LLM topic-labelling call falls back to raw BERTopic names on failure (Chapter 5), and `log_clustering_run` falls back to a logged warning (Section 7.5): every side channel degrades quietly, while the two things each run actually exists to produce, saved articles and labelled topics, either succeed outright or fail loud enough for Prefect’s retry, or the operator watching the logs, to notice.

Put together, News Insight’s operational visibility rests on three surfaces, each already introduced from a different angle in an earlier chapter: the Prefect UI (Figure 7.2) for flow- and task-run history and retries; the MLflow UI (Figure 7.3) for clustering-run parameters and metrics; and the `pipeline_metrics` table (Chapter 4, Section 4.3) as the one destination queryable directly with SQL, outside either tool’s own UI or retention window, for the scrape/transform/load side of the pipeline.

7.7 Conclusion

This chapter closed the operational story every earlier chapter had deferred to it. A single Prefect flow, `news-pipeline`, wraps `run_pipeline` in one retried task, `run-news-pipeline` (two attempts, 60 seconds apart); a registered daily deployment names the schedule (06:00, Africa/Tunis) that the flow is meant to run on, though, as read directly from `docker-compose.yml`, no worker process in the current stack is actually configured to execute it, leaving the manual CLI invocation and the `POST /pipeline/trigger` endpoint as the paths that run today. Every BERTopic run is logged separately, under MLflow’s `news-clustering` experiment, with the same parameters and metrics Chapter 5 introduced and this chapter finally showed being written and displayed. Across both services and the pipeline code they wrap, the same reliability posture repeats: the steps that actually produce data (scraping, transforming, saving, clustering) fail loud, letting Prefect’s retry or an operator’s attention handle it, while every side channel that only records *what happened* (a `pipeline_metrics` row, an MLflow run) fails quiet, so a monitoring outage never becomes a data outage. The next chapter turns from this orchestration layer to the FastAPI backend it and every other feature in this report sit behind (the routes already named in passing across five chapters, from `/pipeline` to `/genai` to `/search`) as

one coherent surface.

BACKEND AND DATA EXPOSURE

Chapter outline

1	Introduction	50
2	Orchestration technology choice	50
3	Flow design	51
4	Scheduling	53
5	Experiment tracking with MLflow	54
6	Monitoring and reliability	56
7	Conclusion	57

8.1 Introduction

Chapter 1’s technology table already gave FastAPI its one-line role in the stack: “exposing articles, search, and GenAI endpoints to the frontend” (Table 1.1), and every chapter since has read a slice of what sits behind one such endpoint without ever stepping back to look at the API as a single surface. Chapter 4 traced the pgvector query behind `GET /search/`’s primary path (Section 4.4); Chapter 6 opened `api/routes/genai.py` and `api/routes/search.py` to describe the LLM calls each endpoint makes (Sections 6.3, 6.5, 6.6); and Chapter 7 named `POST /pipeline/trigger` and `POST /pipeline/cluster` as the two HTTP entry points into the orchestration layer (Section 7.3). This chapter reads the remaining, connective layer none of those chapters covered: `api/main.py`, the single file that creates the FastAPI application, configures CORS, and mounts five router modules, one per domain; `api/routes/articles.py`, the read side of the `articles` table that every dashboard view in the frontend depends on and that no earlier chapter has opened at all; `api/routes/img.py`, the same-origin image proxy behind every article thumbnail; and `etl/load.py`’s `get_connection`, the one function every route handler in the codebase calls to reach PostgreSQL. Together they are the whole of what a client of News Insight (the React dashboard, or anyone reading `/docs` directly) actually sees of the backend.

8.2 Backend technology choice

`requirements.txt` lists `fastapi` and `uvicorn` together under an `# API` heading, with no version pin on either, unlike Prefect’s own explicit `prefect>=3.0.0` (Chapter 7), and `api/Dockerfile`’s final line, `CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]`, is what actually serves the application: FastAPI defines the routes, uvicorn is the ASGI server process that runs them. The choice sits on the same simplicity/capability trade-off this report has already named twice: `requests`+BeautifulSoup over browser automation in Chapter 3, Prefect over Airflow in Chapter 7, weighed here against Flask and Django, the field’s other well-known Python web frameworks. Flask is a comparably small ASGI/WSGI surface, but leaves request validation and API documentation to third-party extensions; Django bundles an ORM and an admin site as first-class citizens, which would sit oddly on top of a data layer this report has already shown, and will show again in Section 8.5, is deliberately unmanaged, hand-written SQL rather than mapped model classes. FastAPI’s own value proposition lines up with what the codebase actually uses it for: Pydantic models double as request-body validation with no extra code, exactly what `genai.py`’s `ChatBody` does for `POST /genai/chat`’s single `q` field (Section 6.6); route signatures typed with plain Python types (`article_id: int, limit: int = Query(20, le=100)`) double as query-parameter

parsing and bounds-checking; and the framework introspects those same signatures to generate a full OpenAPI schema and an interactive Swagger UI at `/docs` with no separate documentation step at all (Figure 8.1). One more feature is native to FastAPI rather than layered on top of a generic WSGI app: `BackgroundTasks.add_task`, the exact mechanism Chapter 7 already named as how `POST /pipeline/trigger` and `POST /pipeline/cluster` fire a long-running job without blocking the HTTP response (Section 7.3), a feature the orchestration chapter depends on that a Flask app would need an added task queue to reproduce.

8.3 API structure

`api/main.py` is short and does exactly three things. First, it instantiates the application: `app = FastAPI(title="News MLOps API", description="API for Tunisian news scraping, clustering and sentiment analysis", version="1.0.0")`. The `title` and `description` are what render at the top of the Swagger page in Figure 8.1, and `version` is a plain string the codebase never reads back anywhere else. Second, it adds `CORSMiddleware` (Listing 8.1) with an explicit, three-origin allowlist: `http://localhost:5173` and `http://127.0.0.1:5173` (the Vite dev server) and `http://localhost:8000` (Swagger UI's own self-calls), `allow_credentials=True`, and both methods and headers left wide open with `"*"`. Third, it mounts five router modules, one per domain, each contributed by its own file under `api/routes/`: `app.include_router(pipeline.router, prefix="/pipeline", tags=["pipeline"])`, and the same call repeated for `articles (/articles)`, `search (/search)`, `genai (/genai)`, and `img (/img)`, the `tags` argument is what groups endpoints into the five labelled sections visible in Figure 8.1's Swagger page. A sixth route, `GET /health`, is defined directly on `app` rather than through any router, returning a bare `{"status": "ok"}`; no `healthcheck` block for the `api` service exists in `docker-compose.yml` the way Chapter 7 described for `postgres` and `prefect` (Section 7.6), so this endpoint is reachable but currently unused by the containerized stack's own liveness checks.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",    # Vite dev (direct access / health checks)
        "http://127.0.0.1:5173",
        "http://localhost:8000",    # Swagger UI self-calls
    ],
    allow_credentials=True,
```

```
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Listing 8.1: the CORS configuration (`api/main.py`), a small, explicit allowlist rather than a wildcard origin.

That allowlist is narrower than it might first appear necessary, because the dashboard’s normal traffic never actually needs it. `frontend/src/api/client.js` builds its `axios` client with `baseURL: '/api'`, a same-origin, relative path in both environments the project ships: in development, `vite.config.js` proxies every `/api/*` request server-side to `http://localhost:8000`, stripping the `/api` prefix before forwarding it; in the Docker Compose stack, `frontend/nginx.conf` does the equivalent job, proxying location `/api/` to `http://api:8000/` over the compose network with a comment that states the consequence plainly: “no CORS, no hardcoded localhost.” Because the browser only ever calls its own origin in both cases, the CORS middleware is not what makes the dashboard work; it is a narrower allowance for exactly the two cases its own comments name: a request made directly against `localhost:8000` rather than through either proxy (manual debugging, a health check), and Swagger UI’s “Try it out” self-calls from the `/docs` page it is itself served from.

8.4 Endpoints

Table 8.1 lists every route the five routers and `app` itself expose, with the prefix each router is mounted under already folded into the path. Two small conventions are worth flagging because they affect the exact URL a client must call: `articles.router` and `search.router` both declare their list-style endpoint at path `"/"`, so the full route carries a trailing slash (`/articles/`, `/search/`), exactly what `frontend/src/api/client.js`’s call sites use (`client.get('/articles/', ...)`, `client.get('/search/', ...)`); `img.router`, by contrast, declares its only endpoint at path `""`, so the mounted route is `/img` with no trailing slash at all, again matching the one place the frontend builds that URL directly, `frontend/src/utils/image.js`’s `‘/api/img?url=$encodeURIComponent(url)’`. Inside `articles.py`, ordering also matters: `get_article`’s catch-all `@router.get("/{article_id}")` is declared last in the file, after the four fixed-path endpoints (`/stats`, `/topics`, `/timeline`, `/regions`). FastAPI matches routes in registration order, so a request for `/articles/stats` resolves to `get_stats` rather than being swallowed by the parameterized route beneath it.

Method	Path	Purpose
GET	/health	Bare liveness probe (<code>{"status": "ok"}</code>); not wired into any <code>docker-compose.yml</code> healthcheck.
POST	/pipeline/trigger	Runs the scrape/transform/load pipeline as a FastAPI background task (Chapter 7, Section 7.3).
POST	/pipeline/cluster	Runs BERTopic clustering as a background task (Chapter 7, Section 7.3).
GET	/articles/	Lists articles; filterable by <code>source</code> , <code>sentiment</code> , <code>topic_id</code> , <code>region</code> , paginated with <code>limit/offset</code> .
GET	/articles/stats	Total article count plus positive/neutral/negative sentiment breakdown.
GET	/articles/topics	Topic distribution (<code>topic_id</code> , <code>topic_label</code> , article count), ordered by count.
GET	/articles/timeline	Daily sentiment counts over the last <code>days</code> days (default 30).
GET	/articles/regions	Per-governorate article count and dominant sentiment for the map dashboard.
GET	/articles/{article_id}	Single article by id, including full <code>content</code> .
GET	/search/	AI-assisted semantic search over articles (Chapters 4 and 6).
GET	/genai/summary/{article_id}	Cached or on-demand LLM article summary (Chapter 6, Section 6.3).
POST	/genai/chat	“Ask the news” retrieval-augmented chat assistant (Chapter 6, Section 6.6).
GET	/img	Same-origin, SSRF-guarded proxy for a remote article image.

Table 8.1: Every HTTP endpoint exposed by the News Insight API, with each router’s mount prefix already folded into the path.

`api/routes/img.py`, named in the table’s last row, has not been opened by any earlier chapter.

Its own module docstring states the problem it exists to solve: some sources, `lapresse.tn` among them,

serve a “forbidden” placeholder image to a cross-site browser request because their WAF keys off the `Sec-Fetch-Site/Referer` headers a direct `` tag sends. `proxy_image(url)` fetches the real image server-side instead, with a same-site `Referer` and none of the cross-site fetch metadata a browser would add, and returns the bytes through the API’s own origin, exactly the URL `frontend/src/utils/image.js` builds for every article thumbnail. Because the endpoint accepts an arbitrary `url` query parameter and fetches whatever it points to, it is guarded against server-side request forgery before any request leaves the container: `parsed.scheme` must be `http` or `https`, and `_is_public_host` resolves the hostname and rejects it if any resolved address is private, loopback, link-local, reserved, multicast, or unspecified, blocking a URL like `http://169.254.169.254/` or `http://localhost:5432/` from ever being fetched on the server’s behalf. A successful fetch is further constrained: the response’s `Content-Type` must start with `image/` or the endpoint returns 415, and the body is streamed in 64 KB chunks with an 8 MB running total, aborting with 413 the moment that cap is crossed rather than buffering an arbitrarily large response in memory first. The successful response itself carries `Cache-Control: public, max-age=86400`, so a browser (or an intermediate cache) only re-fetches a given proxied image once a day.

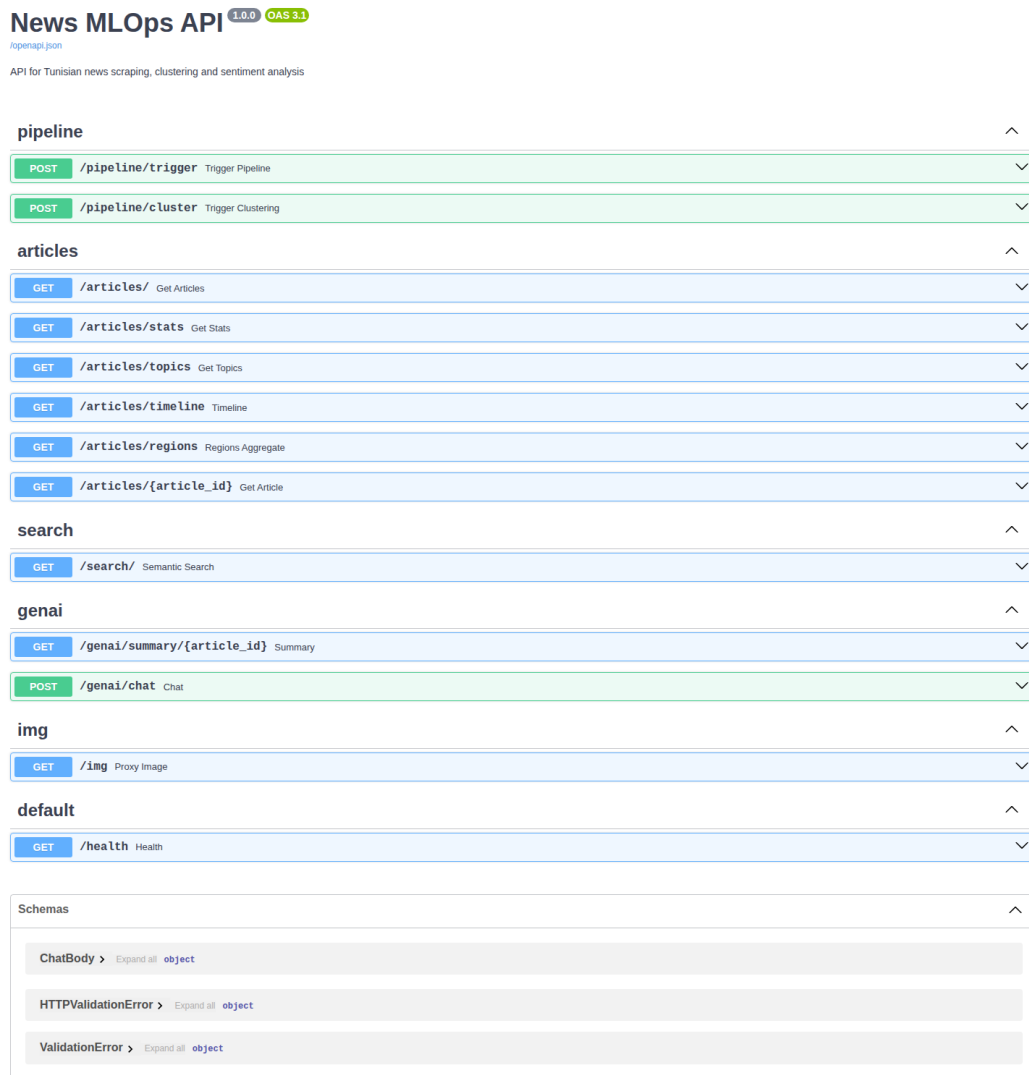


Figure 8.1: FastAPI interactive API documentation (Swagger UI) listing the platform’s endpoints.

8.5 Data access patterns

`requirements.txt` lists `sqlalchemy` under its `# Database` heading alongside `psycopg2-binary` and `pgvector`, but no file in the codebase ever imports it. Every route handler across all five routers reaches PostgreSQL the same way Chapter 2 already named for the ETL layer: `etl/load.py`’s `get_connection()`, a single function that opens a fresh `psycopg2.connect(...)` using `POSTGRES_HOST/POSTGRES_PORT/POSTGRES_DB/POSTGRES_USER/POSTGRES_PASSWORD` read from the environment. There is no model layer between a route handler and the database at all: no declarative table classes, no session object, no query builder: each handler writes its own SQL string directly, with user-supplied values passed as `%s` placeholders that `psycopg2` substitutes safely rather than ever being interpolated into the query text. `get_articles` (`articles.py`) is the clearest illustration of what that looks like in practice: it builds its `WHERE` clause by appending fixed `"column = %s"` fragments from a whitelist

of four known filters (`source`, `sentiment`, `topic_id`, `region`), only for whichever ones the caller actually supplied, so the assembled SQL text always comes from the route's own code, never from a client-controlled string, while the corresponding *values* still travel through parameterized placeholders. The `region` filter carries one special case on top: `region` values of `"national"`, `"null"`, or an empty string are translated to a bare `"region IS NULL"` clause instead of a parameterized equality check, matching the NULL-as-“National / non-localisé”-bucket convention Chapter 5 already described for articles the region tagger could not place (Section 5.6).

Connection lifecycle is handled with one deliberate exception rather than uniformly. Every handler in `articles.py` and `search.py`, and `genai.py`'s `chat`, follow the same shape: open a connection, run one or more queries, call `cur.close()` and `conn.close()` explicitly at the end of the function body, with no `try/finally` guarding those closing calls, so an exception raised by a query itself would skip past them and leave the connection unclosed. `genai.py`'s `summary` is the one handler that does not follow that shape, and its current form is the direct result of two commits: *fix(api): always close DB connection in summary endpoint + guard null content* wrapped the handler's body in `try/finally` so the connection closes on every exit path, including the 404 raised for a missing article; *fix(api): don't hold DB connection across LLM call in summary endpoint* then split that single connection into two short-lived ones: one for the initial `SELECT summary, title, content`, closed *before* the LLM call runs, and a second one opened only for the `UPDATE articles SET summary=%s` that follows, because the earlier version held one connection open, idle, for the entire duration of the LLM request, exactly the failure mode its own commit message names: “pinning a connection idle-in-transaction for the whole LLM latency.” `chat` already avoided that specific problem structurally, since its own retrieval query runs and closes before `complete` is ever called, but it was never given the same `try/finally` guard `summary` received, so of the two GenAI endpoints Chapter 6 covered in depth, only one is hardened against a connection leak on a query-time exception.

8.6 Security and reliability considerations

No endpoint in Table 8.1 requires authentication or an API key; every route, including the two that trigger a pipeline run or a clustering pass (Section 8.4), is reachable by anyone who can reach the API's port at all. That is a workable posture only because of where the API's port is actually reachable from: `docker-compose.yml` exposes it to the host as `"8001:8000"` specifically for direct `curl`/debugging access, with its own comment noting host port 8000 is “often taken by other dev servers,” while the README's own quick-start table still documents `http://localhost:8000/docs`, the container-internal port, correct for `make api`'s local, non-Docker run, but one digit short of the

Compose stack’s actual host mapping. In the containerized deployment the port a browser or client actually needs is never the API’s own: both the Vite dev proxy and `nginx.conf`’s `/api/` location, described in Section 8.3, put a same-origin path in front of it for the dashboard’s own traffic, leaving the direct `:8001` mapping as a debugging convenience rather than the app’s real front door.

Where the API does treat a request as untrusted input, it does so deliberately rather than uniformly: `img.py`’s SSRF guard (Section 8.4) is the one endpoint in the whole surface whose entire design is defensive, because it is also the one endpoint whose entire job is to fetch a URL the client supplies. Every other handler’s only defense is the parameterized-SQL discipline described in Section 8.5; none configures a request timeout of its own on the LLM calls Chapter 6 already described in detail, so `summary` and `chat` inherit whatever default the underlying `openai` client ships with, and a slow or failed provider response blocks that request for as long as the client is willing to wait rather than failing over any faster. That matches the same reliability posture Chapter 6 and Chapter 7 already traced through the rest of the codebase: the steps that produce the actual answer a user is waiting for (an LLM completion, a database write) are left to fail loud rather than silently, while only genuinely optional side effects, an MLflow log line (Chapter 7) or a cached summary column (Section 8.5), degrade quietly. FastAPI itself supplies the one reliability guarantee that costs nothing extra: an unhandled exception anywhere in a handler (a bad query, a missing row not explicitly checked, a provider timeout) becomes a 500 response rather than crashing the whole `uvicorn` worker process, so one bad request never takes the API down for the next one.

8.7 Conclusion

This chapter closed the last piece of News Insight’s architecture no earlier chapter had read end to end: the FastAPI application itself. A single file, `api/main.py`, creates the app, configures a narrow CORS allowlist that the dashboard’s own same-origin proxying in both Vite and `nginx` makes almost unnecessary for its normal traffic, and mounts five domain routers (`pipeline`, `articles`, `search`, `genai`, `img`) whose thirteen endpoints, tabulated in full for the first time in Table 8.1, this report had otherwise only ever named in passing. Underneath every one of them sits the same unmanaged data layer Chapter 4 first described from the schema side: hand-written, parameterized SQL through `etl/load.py`’s `get_connection`, with no ORM in the request path despite one, SQLAlchemy, sitting unused in `requirements.txt`, and a connection-lifecycle fix, traced to two real commits, that hardened exactly one of the API’s LLM-backed endpoints against a leak the other still carries. The next chapter turns from this application layer to how it, and every other service this report has described, is packaged, configured, and run as a single Docker Compose stack.

DEPLOYMENT AND DEVOPS

Chapter outline

1	Introduction	60
2	Backend technology choice	60
3	API structure	61
4	Endpoints	62
5	Data access patterns	65
6	Security and reliability considerations	66
7	Conclusion	67

9.1 Introduction

Chapter 1 named a containerized deployment “that any team member can bring up with a single command” as part of objective O4, and its technology-stack table already gave Docker and Docker Compose their one-line role: “containerized, single-command deployment of the whole stack” (Table 1.1). Every chapter since has read one fragment of that stack from its own vantage point rather than the whole of it: Chapter 4 closed its migrations section by pointing here for how `db/schema.sql` actually runs inside the container (Section 4.5); Chapter 7’s monitoring section already read the `healthcheck` and `depends_on` blocks for `postgres` and `prefect` (Section 7.6) and its experiment-tracking section already named `mlflow`’s host-port remapping (Section 7.5); and Chapter 8’s security section read the `api` service’s own host-port mapping, “8001:8000” (Section 8.6), while its API-structure section traced the `nginx` proxy that lets the dashboard reach that same container without CORS (Section 8.3). This chapter is the first to read `docker-compose.yml` as one whole artifact rather than five separate mentions: all five services it defines, the two Dockerfiles it builds from this project’s own source rather than pulling ready-made (`api/Dockerfile`, `frontend/Dockerfile`), the `.env` file that configures all of them, and the Makefile that wraps the same stack (plus the standalone pipeline scripts it also runs) into the commands the team actually types day to day.

9.2 Containerization strategy

Of the five services `docker-compose.yml` defines, only two carry a `build` block; the other three run an unmodified image pulled straight from a registry, because the whole server each one needs already ships inside that image and none of them embeds any of this project’s own code. `postgres` runs `pgvector/pgvector:pg16`: a PostgreSQL 16 image with the `vector` extension `db/schema.sql` enables already compiled in, sparing the project a custom database image just to add one extension; `prefect` runs `prefecthq/prefect:3-latest`, matching the `prefect>=3.0.0` pin Chapter 7 already read from `requirements.txt` (Section 7.2); `mlflow` runs `ghcr.io/mlflow/mlflow:latest`. `api` and `frontend` are the only two built from a `Dockerfile` in this repository, precisely because they are the only two that carry this project’s own source code rather than a generic server.

`api/Dockerfile` builds on `python:3.11-slim` and installs `build-essential` and `libpq-dev` before anything else: native build dependencies `psycopg2` needs to compile against `libpq` rather than install from a wheel. Its own comments explain the two decisions that follow: `PIP_DEFAULT_TIMEOUT=180` and `PIP_RETRIES=8` are raised because “big ML wheels; slow mirrors time out on the default 15s,” and `torch` is installed in its own separate step, from PyTorch’s CPU-only wheel index, before `requirements.txt` is installed at all, avoiding “the ~2.5GB CUDA wheel stack” a plain `pip`

`install` of the unpinned `torch` dependency in `requirements.txt` would otherwise pull in, on a container that has no GPU to use it. Only after both steps does the image `COPY` the rest of the source tree in and set `CMD` to the same `uvicorn` invocation Chapter 8 already quoted (Section 8.2). `frontend/Dockerfile` instead uses a two-stage build: a first, named stage on `node:20-alpine` runs `npm ci` against the committed `package-lock.json` for a reproducible install, then `npm run build`, producing a static `dist/` directory; a second, final stage on `nginx:alpine` copies only that `dist/` output and `nginx.conf` in, discarding the first stage entirely. The Node.js toolchain that compiled the React app (`node_modules`, the npm CLI, the Node runtime itself) never reaches the image actually shipped and run; the container that serves the dashboard needs nothing more than `nginx` and a directory of static files.

9.3 Service composition

Table 9.1 lists all five services `docker-compose.yml` defines, with the host-side port of each mapping already resolved: the side a developer’s browser or `curl` command actually reaches, as opposed to the container-internal port services address each other by on the compose network. Every service is given an explicit `container_name` under a shared `news_` prefix (`news_postgres`, `news_prefect`, `news_mlflow`, `news_api`, `news_frontend`) rather than the auto-generated name Compose would otherwise assign, and all five carry the same `restart: unless-stopped` policy, a single, uniform choice applied identically whether the service holds state (`postgres`) or none at all (`frontend`).

Service	Image	Host port	Role
postgres	pgvector/pgvector:pg16	5433	Primary datastore: the <code>articles</code> table and pgvector similarity search (Chapter 4).
prefect	prefecthq/ prefect:3-latest	4200	Prefect server (API and UI) backing the <code>news-pipeline</code> flow (Chapter 7).
mlflow	ghcr.io/mlflow/ mlflow:latest	5001	Experiment-tracking UI for BERTopic clustering runs (Chapter 7).
api	built from <code>api/Dockerfile</code>	8001	FastAPI backend; the five routers Chapter 8 traced.
frontend	built from <code>frontend/Dockerfile</code>	5180	React dashboard served by <code>nginx</code> , proxying <code>/api/</code> to <code>api:8000</code> .

Table 9.1: The five services defined in `docker-compose.yml`, with each one’s host-side port mapping.

The two `depends_on` blocks in the file encode a real startup order rather than a cosmetic one. `api` lists both `postgres` and `prefect`, each with `condition: service_healthy`, so `api` only starts once `postgres`’s `pg_isready` check and `prefect`’s own `/api/health` check (Chapter 7, Section 7.6) both report healthy, not merely once their containers exist. `mlflow` is absent from that list entirely, the same asymmetry Chapter 7 already read from the `healthcheck` side: a slow or crashed `mlflow` container never blocks `api` from starting. `frontend` depends on `api` with no condition at all: the default, plain `depends_on` form only waits for the `api` container process to start, not for its `/health` route to actually respond, which matters because, as Chapter 8 already noted, that route is defined directly on the FastAPI app with no corresponding `healthcheck` block for `api` to be waited on by (Section 8.2).

```
api:
  build:
    context: .
    dockerfile: api/Dockerfile
  container_name: news_api
  restart: unless-stopped
```

```
env_file: .env

environment:
  POSTGRES_HOST: postgres
  POSTGRES_PORT: 5432
  PREFECT_API_URL: http://prefect:4200/api
  MLFLOW_TRACKING_URI: http://mlflow:5000

ports:
  - "8001:8000"

depends_on:
  postgres:
    condition: service_healthy
  prefect:
    condition: service_healthy
```

Listing 9.1: the `api` service definition (`docker-compose.yml`): built from source, its environment overridden to compose-network hostnames, and gated on two dependencies’ health rather than their mere existence.

9.4 Environment configuration

`.env.example` is the single template for every configurable value the stack reads, grouped under three headings: database credentials and connection settings (`POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB`, `POSTGRES_HOST`, `POSTGRES_PORT`); the two orchestration and tracking URLs (`PREFECT_API_URL`, `MLFLOW_TRACKING_URI`); and the GenAI provider settings (`LLM_API_KEY`, `LLM_BASE_URL`, an optional `LLM_MODEL` plus the three per-feature overrides `LLM_MODEL_SUMMARY`, `LLM_MODEL_LABELS`, `LLM_MODEL_CHAT` Chapter 6 already described). The README’s own instructions are explicit that this file is copied, not read directly: `cp .env.example .env`, then filled in with real values before anything else is started.

Only two of the five services reference that file at all: `postgres` and `api` both declare `env_file: .env`; `prefect`, `mlflow`, and `frontend` need none of its variables and omit the line entirely. `postgres`’s own `environment` block additionally re-declares `POSTGRES_USER`, `POSTGRES_PASSWORD`, and `POSTGRES_DB` as `$POSTGRES_USER`-style references, a second, distinct mechanism from `env_file`: Compose substitutes those `${...}` placeholders from the same top-level `.env` file at compose-file parse time, so the same three values reach the official `postgres` image’s own expected variable names through variable substitution, layered on top of `env_file` injecting the whole file into the container’s process

environment directly. `api`'s own `environment` block (Listing 9.1) is where the two-context problem Chapter 7 already named for `MLFLOW_TRACKING_URI` (Section 7.5) is actually resolved: `.env.example`'s defaults are written for a process running directly on the host (`localhost` for every host and port), so `api`'s block overrides four of them (`POSTGRES_HOST`, `POSTGRES_PORT`, `PREFECT_API_URL`, `MLFLOW_TRACKING_URI`) to the compose-network service names `postgres`, `prefect`, and `mlflow`. The three `LLM_*` variables are not among them: nothing in `docker-compose.yml` overrides `LLM_API_KEY` or `LLM_BASE_URL`, so the containerized `api` reads the GenAI provider configuration Chapter 6 described straight from `.env`, identically to a `make api` run on the host, only the infrastructure hostnames change between the two, never the LLM provider wiring.

Variable(s)	<code>.env.example</code> default	Overridden for <code>api</code> in Compose
<code>POSTGRES_HOST</code> / <code>_PORT</code>	<code>localhost</code> / <code>5432</code>	<code>postgres</code> / <code>5432</code>
<code>PREFECT_API_URL</code>	<code>http://localhost:4200/api</code>	<code>http://prefect:4200/api</code>
<code>MLFLOW_TRACKING_URI</code>	<code>http://localhost:5000</code>	<code>http://mlflow:5000</code>
<code>LLM_API_KEY</code> / <code>_BASE_URL</code>	Groq free tier (REPLACE_ME key)	not overridden

Table 9.2: Environment variables consumed by the `api` service: which ones Compose rewrites to a container hostname, and which pass through from `.env` unchanged.

9.5 Data persistence

`docker-compose.yml` declares exactly two named volumes, `pgdata` and `mlflow_data`, each mounted by exactly one service: `postgres` mounts `pgdata` at `/var/lib/postgresql/data`, and `mlflow` mounts `mlflow_data` at `/mlflow`: the single directory under which the service's own start command points both its backing store, `sqlite:///mlflow.db`, and its artifact root, `/mlflow/artifacts`, so one volume carries both the run metadata and every logged model artifact Chapter 7 described (Section 7.5). `api` and `frontend` mount no volume at all: both are stateless application containers, rebuilt from source on demand rather than accumulating any data of their own between runs.

`postgres` also carries a third mount that is a bind mount rather than a named volume: `./db/schema.sql:/docker-entrypoint-initdb.d/schema.sql`. This is the mechanism Chapter 4 pointed to when it deferred the question of how the schema it described actually gets created

(Section 4.5): the official `postgres`-family entrypoint executes every `.sql` or `.sh` file found under `/docker-entrypoint-initdb.d/` automatically, but only once, the first time the container starts against a data directory that is still empty. On a fresh `pgdata` volume, this runs `schema.sql` end to end with no manual step at all: the `CREATE EXTENSION IF NOT EXISTS vector`, the `unaccent` extension, and every `CREATE TABLE IF NOT EXISTS` statement Chapter 4 read in detail. Once that volume already holds a previous run’s data, however, the same file is never executed again by the container, by the same official-image convention, so the two additive migration files under `db/migrations/`, which Chapter 4 already described as “meant to be run against an already-deployed database” (Section 4.5), have no Compose service, Makefile target, or README step that actually runs them; applying one against a running `news_postgres` container is left to a manual `psql` session, a step the repository documents nowhere.

The two volumes also survive a stack teardown differently depending on which Makefile target is used. `make db-down` runs a plain `docker-compose down`, which removes containers and networks but leaves `pgdata` and `mlflow_data` intact for the next `make db` or `make db-full`; `make docker-down`, by contrast, runs `docker-compose down -v`, and the `-v` flag deletes both named volumes along with the containers: the difference between pausing the stack and discarding every article and every logged clustering run it holds.

9.6 Developer workflow

The Makefile’s own header comment states the two supported workflows in one line each. The first, and the one its own default targets are built around, keeps only `postgres` in Docker and runs everything else as a local process: `make db` starts just that one service (`docker-compose up -d postgres`), after which `make scrape`, `make transform`, and `make cluster` run the three pipeline scripts directly on the host, `make api` runs `uvicorn` with `--reload` for live code changes, and `make frontend` runs the Vite dev server: the exact sequence the README’s own “Pipeline (step by step)” section documents. Three variables tune the pipeline scripts without editing the Makefile itself: `PAGES` (default 5, how many listing pages `scripts.scrape` fetches), `BATCH_SIZE` (default 16, the transform step’s embedding/sentiment batch size), and `MIN_CLUSTER` (default 3, BERTopic’s minimum cluster size, Chapter 5), each overridable on the command line, for example `make scrape PAGES=20`. `make pipeline` chains `scrape`, `transform`, and `cluster` as one target, the same three steps Chapter 7 was careful to distinguish from the Prefect flow of the same casual name (Section 7.3).

scrape:

```
python -m scripts.scrape --pages $(PAGES)
```

```
transform:

python -m scripts.transform --batch-size $(BATCH_SIZE)

cluster:

python -m scripts.cluster --min-cluster-size $(MIN_CLUSTER)

# Run all three in sequence
pipeline: scrape transform cluster
```

Listing 9.2: the pipeline steps (Makefile): three scripts run directly on the host, chained by `make pipeline`, each tunable through one command-line variable.

The second workflow runs the whole system in containers instead: `make db-full` is `docker-compose up -d` with no service named, bringing up all five without rebuilding any image, appropriate once the `api` and `frontend` images already exist and only their data services need to be (re)started; `make docker-up` instead runs `docker-compose up --build`, forcing both custom images to rebuild from the current source tree first: the target to reach for after a change to `requirements.txt` or the frontend's own `package.json` or source, neither of which `db-full` would pick up. `make install` is the one-time setup step behind either workflow, running `pip install -r requirements.txt` and `npm install` under `frontend/` in sequence. `make logs` deliberately narrows rather than generalizes: it runs `docker-compose logs -f api`, tailing only the one service most likely to need debugging (LLM calls, database queries, pipeline triggers) rather than interleaving all five services' output. `make help` prints a short summary of the targets above; it is, together with the Makefile's own header comment, the closest thing the repository has to written onboarding documentation for a new team member's first day.

9.7 Conclusion

This chapter read `docker-compose.yml` as the single artifact behind every fragment of it earlier chapters had already touched: two images built from this project's own source (`api`, on `python:3.11-slim` with a CPU-only `torch` install ahead of the rest of `requirements.txt`, and `frontend`, a two-stage build that ships only compiled static assets and `nginx`, never the Node toolchain that built them) alongside three vendor images needing no Dockerfile of their own; a health-gated startup order that lets `api` wait on `postgres` and `prefect` while never waiting on `mlflow`; a single `.env` file resolved two different ways depending on whether a variable needs to change between a host process and a container (four

of them do, the LLM provider settings do not); and two named volumes, `pgdata` and `mlflow_data`, that give the stack's only two stateful services persistence across a restart, backed by an initialization mechanism that runs `db/schema.sql` exactly once, on an empty volume, and never again, leaving the two additive migrations Chapter 4 introduced with no automated path onto an already-running database. The Makefile ties both of the workflows this chapter described (pipeline scripts run locally against a single containerized database, or the entire stack run and rebuilt in containers) into the small set of commands a new team member actually needs on their first day. The next chapter turns from how the dashboard is packaged and served to what it actually shows: the React frontend's own views, components, and data-visualization choices.

FRONTEND AND DATA VISUALIZATION

Chapter outline

1	Introduction	69
2	Containerization strategy	69
3	Service composition	70
4	Environment configuration	72
5	Data persistence	73
6	Developer workflow	74
7	Conclusion	75

10.1 Introduction

Chapter 9 closed by handing off from how the dashboard is packaged and served to “what it actually shows: the React frontend’s own views, components, and data-visualization choices.” Earlier chapters had already opened individual frontend files precisely once each, for a single, narrow reason: Chapter 6 read `ChatWidget.jsx` and `ArticleModal.jsx`’s `ArticleSummary` to show how the two live GenAI endpoints degrade on the client when a request fails (Sections 6.3, 6.6); Chapter 8 read `client.js` and `image.js` only far enough to explain why the API’s CORS allowlist barely matters in practice (Section 8.3). This chapter is the first to read `frontend/src` as one whole application: `App.jsx`, the single entry point that mounts a router, a persistent navigation header, and a floating chat widget; the six page components each route renders; the smaller, reusable components those pages share; and the two data-visualization libraries (one for charts, one for a choropleth map) that turn the `articles` table’s columns into something a reader can actually look at. Nothing here is invented for the report: every route path, component name, and library import below is read directly from the source tree as it stands on this branch.

10.2 Frontend technology choice

Chapter 1’s technology-stack table already gave the frontend its one-line role: “React, Vite: Web dashboard for browsing, searching, and chatting with the news corpus” (Table 1.1), and `frontend/package.json` shows exactly what that resolves to. `react` and `react-dom` are pinned to `~19.2.4`, and `vite` to `~8.0.4` as the build tool and, in development, the dev server Chapter 8 already named behind the `/api` proxy (Section 8.3). Vite was chosen over the older Create React App tooling for the same reason Chapter 9’s two-stage `frontend/Dockerfile` exists at all: an esbuild-based dev server that starts and hot-reloads in milliseconds rather than seconds, and a production `npm run build` that outputs a single static `dist/` directory Section 9.2 already described `nginx:alpine` serving with no Node.js runtime left in the shipped image.

Four further dependencies do the rest of the work this chapter describes. `react-router-dom` (`~7.14.0`) is the client-side router behind every route in Section 10.3; `axios` (`~1.15.0`) is the HTTP client `api/client.js` wraps into the single `baseURL: '/api'` instance every page imports; `lucide-react` (`~1.7.0`) supplies every icon glyph used across the Masthead, the stat cards, and the chat widget, imported by name rather than as a sprite sheet; and, specific to data visualization, `recharts` (`~3.8.1`) is the charting library behind every chart in `Dashboard.jsx`, while `d3-geo` (`~3.1.1`) is the projection library behind the governorate map in `RegionMap.jsx`. There is no separate map component library such as `react-simple-maps` or `Leaflet` in the dependency list; the map is

a hand-built SVG projected with `d3-geo`'s low-level `geoIdentity/geoPath` functions, described in Section 10.5. No UI component kit (Material UI, Ant Design, Tailwind) appears in `package.json` either: every page and component instead pairs a `.jsx` file with its own same-named `.module.css` file, scoped per file by Vite's built-in CSS Modules support, on top of a shared set of CSS custom properties (`--brand`, `--positive`, `--neutral`, `--negative`, among others) that ties the hand-rolled design system together. `Dashboard.jsx` even reads several of them directly in JavaScript, in its `TOOLTIP_STYLE` object applied to every recharts `Tooltip`.

10.3 Application structure

`main.jsx` is the one file that actually calls `react-dom`: it mounts a single `<App />` inside `React.StrictMode` at `document.getElementById('root')`, and never changes for the lifetime of the application. `App.jsx` itself is short and declares the whole shape of the site in one component (Listing 10.1): a `BrowserRouter` wraps three siblings: `<Masthead />`, a `<main>` element holding `<Routes>`, and `<ChatWidget />`, so the navigation header and the floating chat assistant are the two pieces of chrome present on every route, mounted once outside the `<Routes>` block rather than repeated inside each page.

```
<BrowserRouter>
  <Masthead />
  <main className={styles.main}>
    <Routes>
      <Route path="/" element={<Reader />} />
      <Route path="/dashboard" element={<Dashboard />} />
      <Route path="/articles" element={<Articles />} />
      <Route path="/search" element={<Search />} />
      <Route path="/carte" element={<Carte />} />
      <Route path="/pipeline" element={<Pipeline />} />
    </Routes>
  </main>
  <ChatWidget />
</BrowserRouter>
```

Listing 10.1: the whole application shell (`App.jsx`): six routes, and two pieces of chrome mounted once, outside the routed content.

Table 10.1 lists all six routes with the page component and content each one owns. `Masthead.jsx` exposes five of them as its own `NavLink` array (À la `une`, `Articles`, `Carte`, `Recherche`, `Analyses`, each paired with a `lucide-react` icon and pointing at `/`, `/articles`, `/carte`, `/search`, `/dashboard` respectively) rendered in a second navigation row below the masthead’s top bar; `/pipeline` is deliberately not among them. It is instead a separate `NavLink` in the top bar itself, rendered as a single `Zap` icon with no visible label, next to the current date, a visual demotion consistent with what the page behind it actually is: not a reader-facing view but an operator control panel for triggering pipeline runs, covered in Section 10.3 only as a route and left otherwise to Chapter 7’s account of what `POST /pipeline/trigger` and `POST /pipeline/cluster` actually do (Section 7.3).

Route	Component	Content
<code>/</code>	<code>Reader.jsx</code>	Home magazine reader: a hero card for the latest article, then a source-interleaved grid with per-source filter chips (Section 10.5).
<code>/dashboard</code>	<code>Dashboard.jsx</code>	Stat cards plus three <code>recharts</code> views (sentiment split, top topics, 30-day sentiment timeline) and a recent-articles strip (Section 10.5).
<code>/articles</code>	<code>Articles.jsx</code>	Full-corpus browser: sentiment, topic, and source filters with offset-based pagination (Section 10.4).
<code>/search</code>	<code>Search.jsx</code>	AI-assisted semantic search over the corpus (Section 10.4; Chapter 6, Section 6.5).
<code>/carte</code>	<code>Carte.jsx</code>	Tunisia governorate map with a volume/sentiment metric toggle and a drill-down side panel (Section 10.5).
<code>/pipeline</code>	<code>Pipeline.jsx</code>	Operator controls to trigger a scrape run or a BERTopic clustering pass, each behind a confirmation dialog (Chapter 7, Section 7.3).

Table 10.1: The six routes `App.jsx` declares, the page component behind each, and where this report covers its content.

A small set of components recurs across most of these pages rather than being rebuilt per view. `ArticleCard.jsx` and `HeroCard.jsx` render an article’s thumbnail (proxied through `utils/image.js`’s `proxied()`, Chapter 8’s SSRF-guarded `/img` route, Section 8.3), title, source, a relative timestamp from `utils/time.js`’s `relativeTime()`, and a `SentimentBadge` in its compact `variant="dot"` form; both fall back to a `lucide-react` `ImageOff` placeholder icon if the image fails to load or the article has

none. `ArticleModal.jsx` is the one component every page but `Pipeline.jsx` mounts on demand to show an article's full text, its cached-or-on-demand LLM summary (Chapter 6, Section 6.3), and a link to the original source. `Reader`, `Dashboard`, `Articles`, `Search`, and `Carte` each keep their own `selectedArticle` state and render the same modal when a card or result is clicked. `PageHeader.jsx` supplies the title-plus-subtitle banner at the top of `Dashboard`, `Articles`, `Search`, `Carte`, and `Pipeline` (`Reader` alone skips it, leading instead with `HeroCard`), and optionally renders arbitrary children as right-aligned actions. `Carte.jsx` is the only page that uses this slot, for its volume/sentiment toggle buttons. `ConfirmDialog.jsx` is used in exactly one place, `Pipeline.jsx`, gating both the scrape trigger and the clustering trigger behind an explicit confirm/cancel step before either destructive-feeling, potentially long-running action fires.

Because `/carte`, `/search`, and every other non-root route only exist as client-side state inside this single-page application, the server has to hand back `index.html` for all of them rather than looking for a matching file: Chapter 9's account of `frontend/nginx.conf` stopped at its `/api/` proxy block (Section 8.3), but the same file's `location /` block is `try_files $uri $uri/ /index.html`, the SPA fallback `react-router-dom` needs to take over regardless of which route was requested first. That fallback was the site of a real bug, `fix/spa-cache-headers` (Chapter 1's branch history): `index.html` was served with no `Cache-Control` header, so a browser cached it and kept referencing an old, deleted JavaScript bundle after a deploy. A route added since, such as `/carte`, then had no matching code and rendered blank. The fix serves `index.html` as `no-cache, must-revalidate` while the hashed assets alongside it stay `immutable` for a year: the shell naming the current routes is always revalidated, the bundles it points to are cached forever.

10.4 Article exploration and search UX

`Articles.jsx` is the corpus browser proper: a filter bar (a `sentiment` `<select>`, a `topic_id` `<select>` populated from `GET /articles/topics`, and a free-text `source` `<input>`) feeds a single `fetchArticles` callback that calls `GET /articles/` with whichever of `source`, `sentiment`, and `topic_id` are non-empty, plus `limit=20` and an `offset` computed from the current page. Changing any filter resets `page` to 0 so a stale offset from a previous filter never silently drops results. Pagination itself works around a real gap in the API: Table 8.1 showed that `GET /articles/` returns a page of rows with no accompanying total count, so `Articles.jsx` estimates one client-side. Its own comment states the rule plainly, "if backend doesn't return total, estimate from results": a full page (twenty rows) assumes at least one more page exists ($(\text{page} + 2) * \text{PAGE_SIZE}$), and a short page sets the total to exactly what has been seen so far. The estimate can only ever overshoot by up to one page, never undershoot,

so the “Page n ” indicator and the disabled state of the forward arrow (`disabled={articles.length < PAGE_SIZE}`) stay consistent with what the user can actually click through.

`Search.jsx` is the frontend half of the AI-assisted semantic search Chapter 6 covered in full from the backend side (Section 6.5): a single input plus a “Rechercher” button call `GET /search/` with the trimmed query and `limit=20` on submit, showing a `lucide-react Loader2` spinner in place of the button label while the request is in flight. Each result renders a `SentimentBadge`, the source name, the title, and, when the API response carries a `score` or `similarity` field, a percentage figure alongside a horizontal progress bar whose width is set directly from that score, the same numeric relevance signal Chapter 6 traced back to pgvector cosine distance or, on the LLM-ranking fallback path, the candidate’s position in the model’s returned ordering. Clicking a result opens the same `ArticleModal` every other page uses. Figure 10.1 shows this view after a query for “football”: seven results, each carrying its own sentiment badge and source, with no unrelated recency-padded articles mixed in, the visible effect of the search-precision fix Chapter 6 described in detail (Section 6.5).

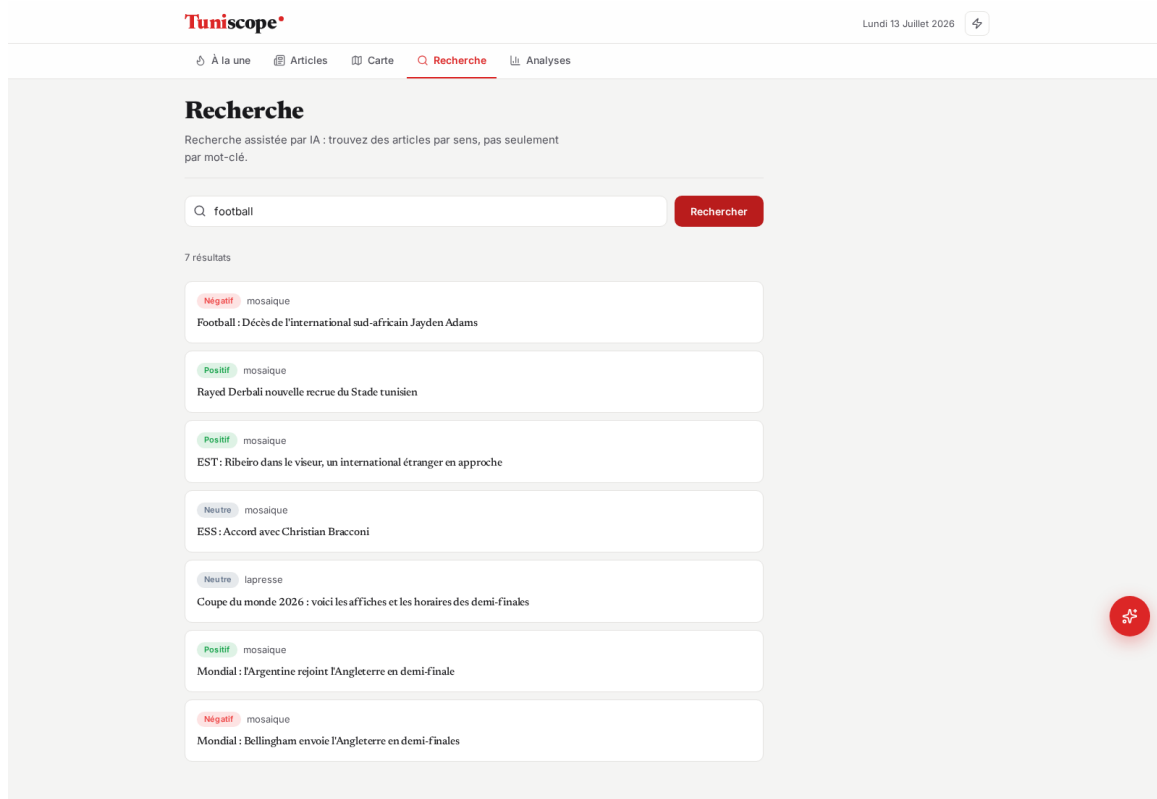


Figure 10.1: The AI-assisted semantic search view (`Search.jsx`): each result carries a sentiment badge, source, title, and relevance score.

10.5 Visualizations and source diversity

`Reader.jsx`, the site’s home route, is the first place a reader’s exposure to source diversity is engineered rather than left to whatever order `GET /articles/` happens to return. Its `interleaveBySource` helper

(Listing 10.2) groups the sixty most recent articles into one queue per source, then round-robins across those queues, pushing one article from each in turn until every queue is empty. The function’s own comment states why: “so À la une shows a mix, not just the source with the most recent scrape batch.” Without it, a burst of scraping from a single outlet would crowd out every other source from the first screen a reader sees. The resulting list feeds a hero card (the first article) and a grid of the rest, with per-source filter chips computed from the same interleaved set (`[...new Set(articles.map(a => a.source))]`) plus a “Toutes” catch-all tab, so a reader can narrow back down to a single outlet after the homepage has already shown a deliberately mixed first impression. Figure 10.2 shows this view: a hero article, then a source-filtered grid (Toutes, mosaïque, lapresse, kapitalis) directly below it.

```
function interleaveBySource(list) {
  const bySrc = new Map()
  for (const a of list) {
    if (!bySrc.has(a.source)) bySrc.set(a.source, [])
    bySrc.get(a.source).push(a)
  }
  const queues = [...bySrc.values()]
  const out = []
  let added = true
  while (added) {
    added = false
    for (const q of queues) {
      if (q.length) { out.push(q.shift()); added = true }
    }
  }
  return out
}
```

Listing 10.2: round-robin source interleaving (`interleaveBySource`, `Reader.jsx`): one article per source per pass, so no single outlet crowds out the rest of the homepage.

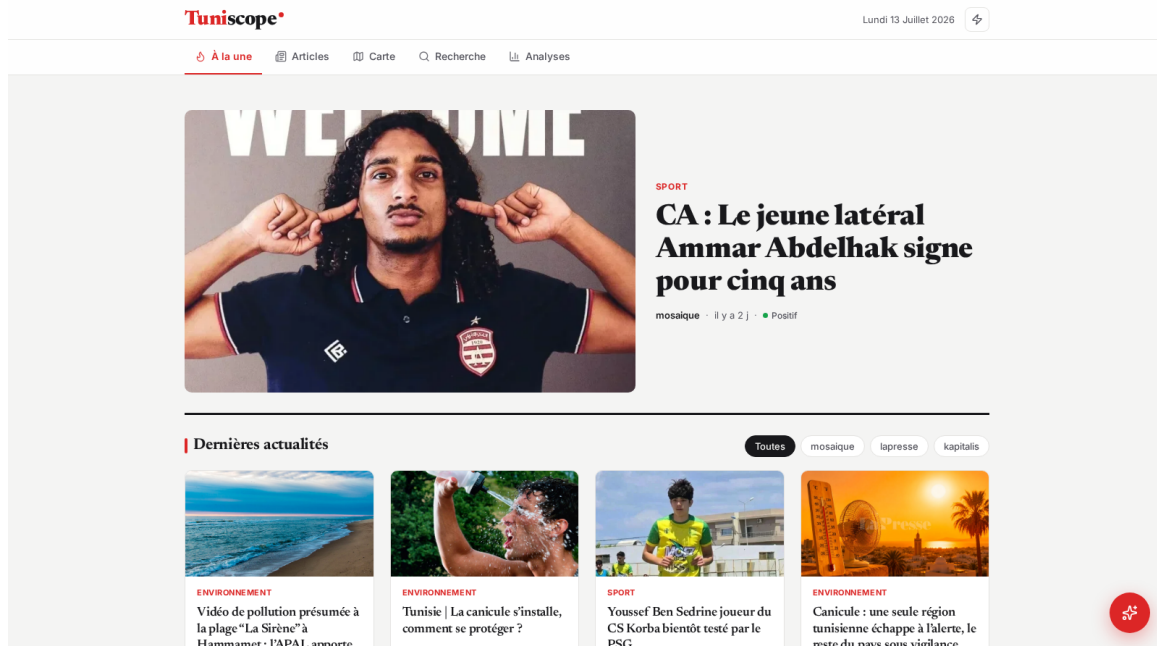


Figure 10.2: The home reader view (`Reader.jsx`, route `/`): the hero card for the latest article, followed by the source-interleaved “Dernières actualités” grid and its per-source filter chips.

`Dashboard.jsx`, mounted at `/dashboard`, is where `recharts` does the rest of the corpus’s diversity justice numerically. On mount it fires four requests in parallel with `Promise.all`: `/articles/stats`, `/articles/` (`limit=5`, for a recent-articles strip), `/articles/topics` (kept to the top ten), and `/articles/timeline` (`days=30`), and renders three charts from the results. A `PieChart` with a hollow `innerRadius` (a donut) shows the positive/neutral/negative split from `/articles/stats`, each slice’s `Cell` colored from `SENTIMENT_COLORS` and labelled with its French name and percentage. A horizontal `BarChart` (`layout="vertical"`) shows the ten most common topics by `topic_label`, the same LLM-assigned themes Chapter 6 described (Section 6.4), each bar’s category axis truncating a long label to fourteen characters plus an ellipsis. A `LineChart` plots three separate `Line` series (positive, neutral, negative) against `date` over the last thirty days, the frontend rendering of the daily sentiment aggregation Chapter 5’s `etl/metrics.py` (or the equivalent `/articles/timeline` query) computes. All three charts are wrapped in a `ResponsiveContainer` and share the same `TOOLTIP_STYLE` object described in Section 10.2, so a hover tooltip on any of the three looks identical regardless of chart type.

`Carte.jsx`, mounted at `/carte`, is the geographic counterpart: a choropleth of Tunisia’s 24 governorates built from `regions.py`’s gazetteer (Chapter 5, Section 5.6) rather than any third-party map component. `RegionMap.jsx` imports `geoIdentity` and `geoPath` from `d3-geo` and projects `frontend/src/assets/tunisia-governorates.json` (a GeoJSON file whose `properties.slug` values match the same slugs `regions.py`’s `GOVERNORATES` gazetteer assigns each governorate’s `geojson_id` to) into an SVG `<path>` per governorate, using a plain planar identity projection with `reflectY(true)` rather than `geoMercator`, because, per the component’s own comment, a geographic projection

“treats geoBoundaries’ ring order as the globe’s complement and collapses the map” on this particular GeoJSON’s winding order. `Carte.jsx` itself fetches `GET /articles/regions` once and toggles between two coloring metrics that `RegionMap.jsx`’s `fillFor` function switches on: a fixed color per dominant sentiment, or a single brand-red fill whose alpha channel scales with a governorate’s article count relative to the busiest one. Clicking a governorate (or the “National / non-localisé” button for articles `regions.py` could not place) opens `RegionPanel.jsx`, which fetches up to eight articles for that region via `GET /articles/` with a `region` filter, renders a stacked sentiment bar, and reuses `ArticleCard` for the list.

That article list was itself the subject of a real, documented bug: `fix/region-panel-scroll` (Chapter 1’s branch history again) found that clicking a governorate with many articles scrolled the entire page rather than the panel, with every `ArticleCard` inside rendering no image at all. Both symptoms traced to one CSS interaction: the panel’s article list is a flex column, and with enough cards its flex children were shrunk to fit, collapsing each card’s `overflow: hidden` image wrapper down to zero pixels while leaving the list short enough that the page scrolled instead of it. The fix (`flex-shrink: 0` on the list’s children, the panel itself made `position: sticky` and capped to the viewport, and `overscroll-behavior: contain` on the list) is three CSS properties, none touching `RegionPanel.jsx`’s JavaScript, resolving both the scroll target and the missing images from a single shared root cause.

10.6 Floating chat assistant

Chapter 6 already traced `POST /genai/chat`’s retrieval-augmented-generation loop and the general shape of `ChatWidget.jsx`’s rendering in detail (Section 6.6); this section is only concerned with where that component sits in the application built in this chapter. As Listing 10.1 showed, `<ChatWidget />` is mounted once in `App.jsx`, as a sibling of `<Routes>` rather than inside any one page, the one piece of UI, alongside `Masthead`, present identically on all six routes. It renders as a fixed, round floating action button in the bottom-right corner, toggling between a `Sparkles` icon and an `X` icon as its `open` state flips; opening it reveals a docked panel rather than navigating anywhere, so a reader can ask a question without losing whatever page, filter, or map selection they were on underneath it. This placement is the outcome of a documented redesign, commit *feat: AI search, LLM themes, source diversity, floating chat assistant*, which removed a dedicated `/chat` route and its `Chat.jsx` page entirely: “Remove the nav tab + `/chat` page. Add a bottom-right FAB that opens a chat popup (`ChatWidget`), available on every page”, the same commit that added `interleaveBySource` to `Reader.jsx` (Section 10.5) and the AI-search precision work Chapter 6 covered (Section 6.5), all three shipped together as one feature-level

change to the frontend.

The accessibility details the commit message called out are visible directly in the component: an **Escape** key handler closes the panel from anywhere, focus moves to the message input on open, and every interactive element carries an explicit **aria-label**. An empty conversation shows three canned French suggestion buttons that call the same **ask()** function a typed question would; a message list alternates user and assistant bubbles, auto-scrolls to a trailing sentinel on every update, and renders an assistant reply's **sources** array (Chapter 6's five retrieved articles) as a bulleted list of external links. The same client-side degrade-on-failure behavior Chapter 6 already traced (an error bubble in French rather than a stalled spinner) is what keeps this always-available widget from ever blocking whatever page it floats over.

10.7 Conclusion

This chapter read **frontend/src** as the single application Chapter 9 handed off to: a React 19 single-page app built and served by Vite, routed client-side by **react-router-dom** across six routes (Table 10.1), with **Masthead** and **ChatWidget** mounted once, outside those routes, so navigation and the ask-the-news assistant are identical on every page. A small, hand-rolled component set (**ArticleCard**, **HeroCard**, **ArticleModal**, **PageHeader**, **SentimentBadge**, **ConfirmDialog**) is reused across most of the six pages rather than rebuilt per view, styled through CSS Modules and a shared set of CSS custom properties rather than a component-kit dependency. Two data-visualization libraries turn the **articles** table into something a reader can read at a glance: **recharts** drives **Dashboard.jsx**'s sentiment donut, topic bar chart, and 30-day sentiment timeline, and **d3-geo**'s low-level **geoIdentity/geoPath** functions drive **RegionMap.jsx**'s hand-built choropleth of Tunisia's 24 governorates, with no third-party map or chart-kit dependency doing the projection work for either. Two real, documented bugs (a stale cached **index.html** breaking client-side routes after a deploy, and a flex-layout interaction that broke both scrolling and images in the same region panel) are further evidence, alongside Chapter 6's own two fixes, that this frontend was iterated against a running system rather than written once and left alone. The final chapter steps back from every individual layer this report has traced (scraping, storage, machine learning, orchestration, the API, and this frontend) to summarize what the finished platform achieves against Chapter 1's original objectives, and what remains open for future work.

FINAL CONCLUSION AND FUTURE IMPROVEMENTS

Chapter outline

1	Introduction	78
2	Frontend technology choice	78
3	Application structure	79
4	Article exploration and search UX	81
5	Visualizations and source diversity	82
6	Floating chat assistant	85
7	Conclusion	86

11.1 Final conclusion

Chapter 1 opened this report with a fragmented Tunisian news landscape (*La Presse*, *Business News*, *Kapitalis*, and *Mosaiïque FM* publishing independently, with no shared index, no cross-source view, and no machine-readable signal attached to any of it) and four concrete objectives to close that gap: O1, automated collection; O2, NLP enrichment; O3, GenAI access; and O4, reproducible MLOps (Table 1.1). Ten chapters later, the platform this report has traced end to end meets all four, each to a degree this closing chapter can now state precisely rather than as an aspiration. O1 runs today through `RSSScraper` polling four sources every time the pipeline executes, with `KapitalisScraper`’s deeper HTML crawl built, tested, and deliberately left dormant (Chapter 3, Section 3.4). O2 runs as a single batched pass over every new article (multilingual e5 embeddings, XLM-RoBERTa sentiment, a ten-category LLM theme, and a gazetteer region tag), with BERTopic clustering as a separate, operator-triggered stage over the same embeddings (Chapter 5). O3 is live in three places a reader actually touches: cached on-demand summaries, LLM-assisted search ranking, and a five-source retrieval-augmented chat assistant (Chapter 6). O4 is real but partial, and this report has been explicit about exactly where: a Prefect flow and a logged MLflow experiment exist and work, but the registered daily deployment has no worker to execute it, and a fifth, supporting objective (a web dashboard usable by someone who does not read code) is fully delivered as the six-route React frontend of Chapter 10. None of these four assessments is asserted from a plan or a design document; each is read directly from the code, the configuration, and the commit history this report has quoted throughout, including the places where what was actually built falls short of what was intended.

11.2 Project contributions

Layer by layer, the platform delivers a working, end-to-end pipeline rather than a set of disconnected proofs of concept. The **collection** layer (Chapter 3) contributes a shared `BaseScraper` template implemented twice (an HTML listing crawler and an RSS collector), converging on one validated `Article` shape, with layered error handling (per-request, per-article, per-source) and a durable, database-enforced uniqueness guarantee on `url`. The **storage** layer (Chapter 4) contributes a single PostgreSQL instance, extended with pgvector, holding both an article’s relational metadata and its 384-dimension embedding in one table, indexed for the exact access patterns the API and dashboard exercise, and grown to its current shape through two small, additive migrations rather than a heavyweight migration framework. The **machine learning** layer (Chapter 5) contributes three model families working together (multilingual sentence embeddings, fine-tuned sentiment classification, and unsupervised BERTopic clustering), each wrapped by a lighter, explicitly-fallback-guarded heuristic (an

LLM theme classifier, an LLM topic labeller, a gazetteer-first region tagger) so a single model failure degrades one column's quality rather than the whole run. The **GenAI** layer (Chapter 6) contributes a single fifty-line provider abstraction, `etl/llm.py`, underlying three reader-facing features (cached article summaries, precision-tuned semantic search, and a five-source "ask the news" chat assistant), all swappable to a different large language model provider through three environment variables. The **orchestration** layer (Chapter 7) contributes a scheduled, retried Prefect flow and a fully populated MLflow experiment for every clustering run, with a documented reliability posture that lets side channels (a metrics row, a tracking log) fail quietly while the steps that produce data fail loud. The **backend** layer (Chapter 8) contributes a five-router FastAPI surface, thirteen endpoints tabulated for the first time in this report, generating its own interactive documentation with no separate step. The **deployment** layer (Chapter 9) contributes a five-service Docker Compose stack, two of its five images built from this project's own source, brought up with a single command either as a fully containerized stack or as a lighter host-plus-database workflow. The **frontend** layer (Chapter 10) contributes six routed views sharing a persistent navigation header and a floating chat assistant, including a hand-built Tunisia governorate choropleth requiring no third-party map library at all. Across every one of these eight layers, this report has also named real, documented bugs found and fixed against a running system: a scraper mis-selecting the wrong feed field, a search-ranking model padding irrelevant results, a database connection pinned idle across an LLM call, a flex-layout interaction breaking both scroll and images in the same panel, evidence that News Insight was iterated against its own behaviour rather than written once from a specification and left alone.

11.3 Future improvements

Every chapter in this report has been as explicit about what is not yet done as about what is. Collected here, the concrete, verified gaps this platform still carries define the next iteration's backlog:

- **Broader source coverage.** Collection today is limited to the four RSS feeds already named: *lapresse*, *businessnews*, *kapitalis*, and *mosaique* (`RSS_FEEDS`, `etl/extract.py`), with no other Tunisian outlet wired in. `KapitalisScraper`'s deeper HTML crawl (Chapter 3, Section 3.4) is fully implemented and tested but sits behind a commented-out line in `build_scrapers`; reactivating it, and writing equivalent listing crawlers for additional outlets, is the most direct way to widen the corpus beyond what RSS alone exposes.
- **A worker for the scheduled pipeline.** Chapter 7 (Section 7.4) traced a real operational gap rather than a hypothetical one: `flows/schedule.py` registers a `daily-news-pipeline` deployment against a cron schedule, but no Prefect worker or work pool is defined anywhere

in `docker-compose.yml` to actually execute it, so today the pipeline only runs through the two direct paths: a manual CLI invocation or `POST /pipeline/trigger`. Adding a worker service to the Compose stack, polling the registered work pool, would let the deployment already defined in code run unattended for the first time.

- **An automated migration runner.** Chapter 9 (Section 9.5) showed that `db/schema.sql` only ever runs once, on an empty `pgdata` volume, through the official Postgres image's initialization convention; the two additive files under `db/migrations/` have no Compose service, Makefile target, or documented step that applies them against an already-running database, leaving that step to an undocumented manual `psql` session. A small migration-runner service, or a lightweight tool such as Flyway or a custom versioned-migrations script wired into the Makefile, would close that gap.
- **Uniform connection handling in the API.** Chapter 8 (Section 8.5) traced a connection-lifecycle fix applied to exactly one endpoint, `GET /genai/summary/{article_id}`, after a real idle-in-transaction bug; every other handler in `articles.py`, `search.py`, and `genai.py`'s `chat` still closes its connection with no `try/finally` guard, and no handler anywhere pools connections rather than opening a fresh `psycopg2.connect(...)` per request. Introducing a shared connection pool (`psycopg2.pool` or an async driver) and applying the same `try/finally` discipline uniformly would remove both the leak risk and the per-request connection overhead at once.
- **Arabic-language content and Arabic-dialect sentiment.** Chapter 1 named that some of the same four outlets already publish in Arabic alongside French, but every entry in `RSS_FEEDS` is tagged "`fr`", and `cardiffnlp/twitter-xlm-roberta-base-sentiment` (Chapter 5, Section 5.4), while multilingual, was neither selected nor validated for Tunisian Arabic or its *Derja* dialect. Ingesting Arabic-language articles and either validating or fine-tuning a dialect-aware sentiment model would extend real coverage rather than only the multilingual embedding model's nominal capability.
- **Enabling the LLM region resolver.** Chapter 5 (Section 5.6) and Chapter 2 both noted that `tag_region_llm` is fully implemented and tested in `etl/regions.py` but never exercised: `transform_articles` calls `tag_region` with `use_llm=False`, so an article the gazetteer cannot place unambiguously is left `region = NULL` rather than falling through to the LLM tier. Turning that flag on, at least for articles the gazetteer left unresolved, would shrink the "National / non-localisé" bucket the map dashboard currently falls back to.

- **Named-entity extraction.** Chapter 4 (Section 4.3) described the `entities` table (one row per extracted named entity, cascading on its parent article) as forward-provisioned schema with no pipeline step that ever inserts into it. Wiring a lightweight named-entity recognition step (a spaCy French pipeline, already a project dependency for BERTopic’s stop-word list, or an LLM-based extractor following the same pattern as theme classification) into `transform_articles` would put that table to its intended use.
- **Authentication for operator and pipeline endpoints.** Chapter 8 (Section 8.6) was explicit that no endpoint in the API requires authentication or an API key, including `POST /pipeline/trigger` and `POST /pipeline/cluster`, both reachable by anyone who can reach the API’s port, currently workable only because that port is not the dashboard’s real front door. Adding at least a shared operator token or API key on the pipeline and clustering routes, ahead of any future deployment where the API’s port is more directly exposed, would close that gap before it becomes a real incident rather than a documented risk.

11.4 Closing remarks

None of the eight gaps above was discovered by inspection after the fact; each was named, in the chapter where it belongs, at the moment the code that has the gap was itself being read. That is a deliberate property of this report and, more importantly, of the project it describes: a three-person, one-semester academic team building a Groq free-tier, single-database, five-service platform under real time and cost constraints, choosing a coarse-grained Prefect task over one task per pipeline step, a gazetteer before an LLM fallback, and a hardened connection-close path for one endpoint before generalizing it to five, trade-offs made deliberately, and documented as such, rather than found later as unexplained shortcuts. News Insight, as it stands at the end of this report, turns four independent, French-language Tunisian news sources into one continuously updated, semantically searchable, sentiment- and topic-tagged corpus, explorable through a chat assistant that cites its sources and a governorate map built without a mapping library, while carrying, openly, the list above as exactly what a next iteration should take on first.

Webography

- FastAPI. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
- Prefect Technologies, Inc. *Prefect Documentation*. <https://docs.prefect.io/>
- MLflow Project, Linux Foundation AI & Data. *MLflow Documentation*. <https://mlflow.org/docs/latest/index.html>
- Maarten Grootendorst. *BERTopic Documentation*. <https://maartengr.github.io/BERTopic/>
- Hugging Face. *Sentence Transformers Documentation*. <https://www.sbert.net/>
- pgvector contributors. *pgvector: Open-Source Vector Similarity Search for Postgres*. <https://github.com/pgvector/pgvector>
- PostgreSQL Global Development Group. *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
- Meta Platforms, Inc. *React Documentation*. <https://react.dev/>
- Vite core team. *Vite Documentation*. <https://vite.dev/>
- Docker Inc. *Docker Compose Documentation*. <https://docs.docker.com/compose/>
- Groq Inc. *GroqCloud API Documentation*. <https://console.groq.com/docs/>
- Kurt McKee et al. *feedparser Documentation*. <https://feedparser.readthedocs.io/>
- Leonard Richardson. *Beautiful Soup Documentation*. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>
- Explosion AI. *spaCy Documentation*. <https://spacy.io/>
- Recharts Group. *Recharts Documentation*. <https://recharts.org/>
- D3 contributors. *d3-geo Documentation*. <https://d3js.org/d3-geo>

Abstract

News Insight scrapes Tunisian news sources, stores them in PostgreSQL with pgvector, and analyses them with embeddings, sentiment analysis, and BERTopic topic modeling.

The platform adds a GenAI layer (summaries, an ask-the-news chat assistant, LLM-generated topic labels) on top of an MLOps stack built with Prefect, MLflow, FastAPI, React, and Docker.

Keywords: News Analysis, MLOps, Web Scraping, NLP, BERTopic, Sentence Transformers, pgvector, FastAPI, Prefect, MLflow, GenAI, RAG